

Deep Analytics Risk Management with AI

Brian Huge & Antoine Savine
Superfly Analytics, Danske Bank

Superfly

Analytics

Least-Square Monte-Carlo (LSM)

- Classic algorithm widely implemented in pricing and risk systems
- Published in the late 1990s by Longstaff-Schwartz and Carriere
Applied for Bermudan options in the Libor Market Model implemented with Monte-Carlo
- Today, LSM would be classified as a Machine Learning (ML) algorithm
The main idea is to learn pricing functions from simulated data
- Repurposed after 2008 for regulations like XVA
LSM learns the value function of an entire trading book

Least-Square Monte-Carlo (LSM)

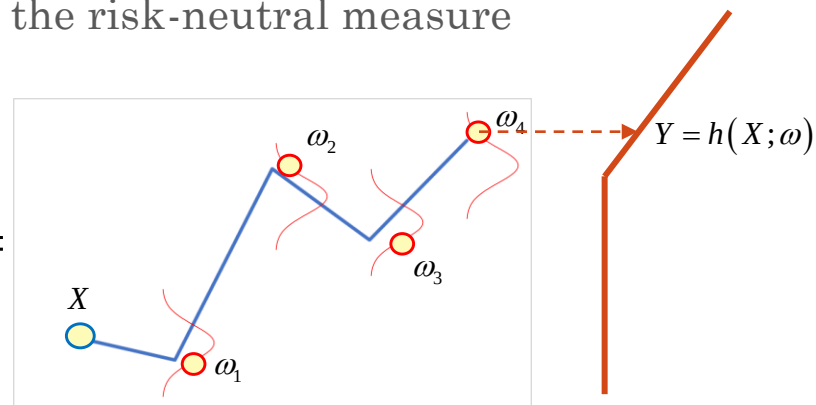
- Synthetic data generated by Monte-Carlo simulations under the risk-neutral measure

- Inputs X are the initial states

- Labels Y are the final (discounted) payoffs

- Random vector ω independent

$$Y = h(X; \omega) =$$



- Inputs X and noise ω are sampled and labels Y are computed with “path-wise payoff” function h

- Encoded in the simulation system

- Depends on the pricing model and cash-flows of contract / trading book

- For example, European call in Black & Scholes: $Y = h(X; \omega) = (S_T - K)^+ = \left(X \exp \left[-\frac{\sigma^2}{2} T + \sigma \sqrt{T} N^{-1}(\omega) \right] - K \right)^+$

Least-Square Monte-Carlo (LSM)

- Dataset $\{x^{(i)}, y^{(i)}\}$ = m independent realisations of the risk-neutral distribution (X,Y)
- Main application:
learn pricing function $v(x) = E[Y|X=x]$ because data is simulated under risk-neutral measure
 - By “training” a universal approximator $\alpha(x; \theta) \approx v(x)$ such as basis function regression or neural network
 - We get a **fast** approximation of the pricing function, which enables:
 - In the context of callable Derivatives: efficient computation of continuation value for pricing/risk by MC
 - For regulations like XVA or back/forward-testing: efficient revaluation of the trading book resolves computation bottleneck
 - Avoid nested MC simulations and other expensive/intractable revaluations
- Also identify principal risk factors and compute risk heat maps with algorithms derived from PCA

Classic Supervised Learning

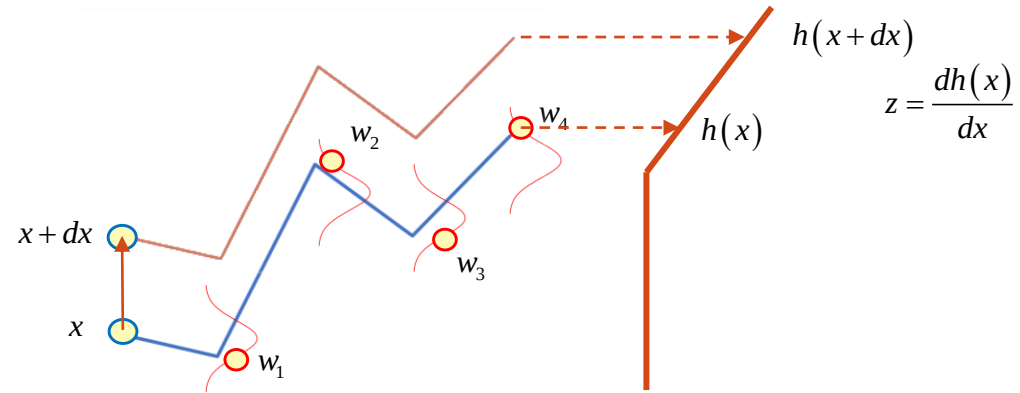
- Dataset $\{X^{(i)}, Y^{(i)}\}$
 - Of m training examples $X^{(i)} \in \mathbb{R}^n$: inputs $Y^{(i)} \in \mathbb{R}$: labels
 - Usually assumed m independent realizations of a joint distribution (X, Y)
 - Typical ML context: pick data in the real world and label by hand
 - Frequently in finance: simulate data and compute labels, as with LSM
- “Train” ML models by minimization of a cost function to:
 - Learn function that predicts labels of unseen inputs $\alpha(x) \approx E[Y|X = x]$
 - Identify all sort of patterns in data

Differential Machine Learning

- Dataset $\{X^{(i)}, Y^{(i)}, Z^{(i)}\}$ m independent realizations of the random variables X, Y, Z
- $Z^{(i)} \in \mathbb{R}^n$: *differential labels* Z is the gradient of Y wrt X: $Z^{(i)} = \frac{\partial Y^{(i)}}{\partial X^{(i)}}$
- i.e. Z = sensitivity of label Y wrt small changes of inputs $x = (x_1, x_2, \dots, x_n)$, everything else constant
- Example: European call in Black & Scholes:

$$Y = h(X; \omega) = \left(\underbrace{X \exp\left[-\frac{\sigma^2}{2}T + \sigma\sqrt{T}N^{-1}(\omega)\right]}_{S_T} - K \right)^+$$

$$Z = \frac{\partial h}{\partial X}(X; \omega) = \underbrace{1_{\{S_T > K\}}}_{S_T / X = S_0} \exp\left[-\frac{\sigma^2}{2}T + \sigma\sqrt{T}N^{-1}(\omega)\right]$$



Obtaining differential labels

- This talk is about what we can do with differential data Z
- First, let us see how it is obtained in the first place
 - When labels Y are obtained by a **computation** as in LSM: $Y = h(X, \omega)$
 - Differential labels Z can be obtained by differentiation: $Z^{(i)} = \frac{\partial h}{\partial X}(X^{(i)}, \omega^{(i)})$
 - E.g. with finite differences: repeatedly bump one $X_j^{(i)}$ and recompute h
 - Very inefficient: costs n evaluations of h
 - **dimension n is usually high**: think Libor Market Model (LMM) for rates or multi-asset models for trading books
 - h is expensive: path simulation + payoff evaluation
 - we need another way

Automatic Adjoint Differentiation

- AAD is a differentiation algorithm that precisely shines in this situation
 - Given a scalar function h of a high-dimensional input x
 - AAD computes the gradient dh/dx analytically, for about 2-3 times the cost of computing h
 - Irrespective of dimension n
- AAD is a powerful algorithm
 - Called 'holy grail' of algorithmic differentiation
 - Classified among the most powerful algorithms of the 20th century
 - Generalizes backpropagation, which powers deep learning
- In finance, AAD was introduced by Giles and Glasserman in 2006 and enabled:
 - Real-time risk reports
 - Instantaneous calibration
 - Novel research like differential ML

AAD

- In return, AAD is not an easy algorithm
 - Takes a bit of work to grasp
 - And a lot of work to implement correctly
- AAD works with computation graphs

AAD

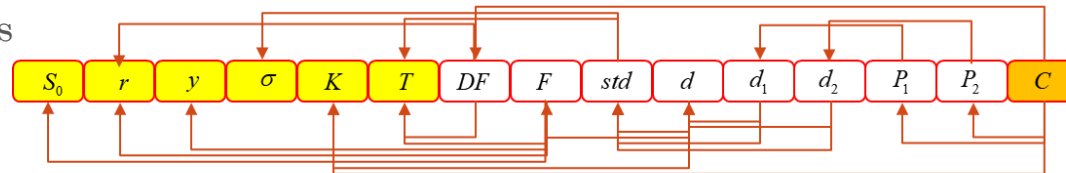
- In return, AAD is not an easy algorithm
 - Takes a bit of work to grasp
 - And a lot of work to implement correctly
- AAD works with computation graphs
 - Given a computation program

```
double blackScholes(  
    const double spot,  
    const double rate,  
    const double div,  
    const double vol,  
    const double strike,  
    const double maturity)  
{  
    double df = exp(-rate * maturity),  
           fwd = spot * exp((rate - div) * maturity),  
           std = vol * sqrt(maturity);  
    double d = log(fwd / strike) / std;  
    double d1 = d + 0.5 * std, d2 = d - 0.5 * std;  
    double p1 = normalCdf(d1), p2 = normalCdf(d2);  
    return df * (fwd * p1 - strike * p2);  
}
```

AAD

- In return, AAD is not an easy algorithm
 - Takes a bit of work to grasp
 - And a lot of work to implement correctly
- AAD works with computation graphs
 - Given a computation program
 - AAD builds the computation graph
 - In topological order by recording operations on a data structure called “tape”

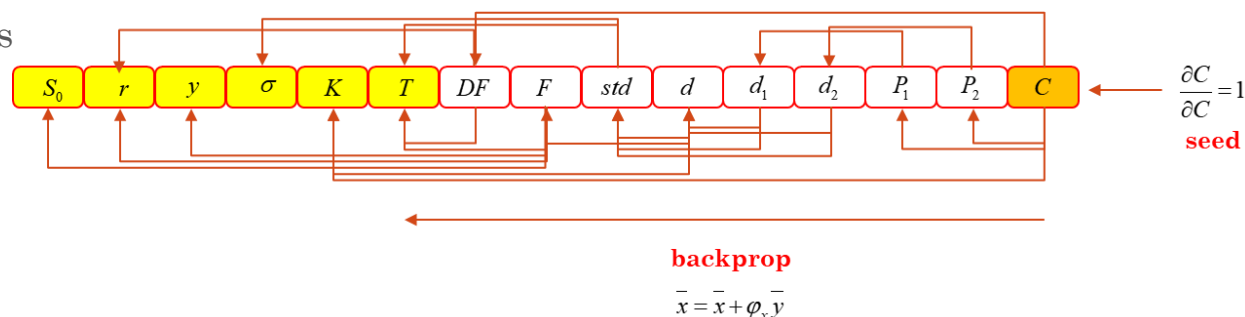
```
double blackScholes(  
    const double spot,  
    const double rate,  
    const double div,  
    const double vol,  
    const double strike,  
    const double maturity)  
{  
    double df = exp(-rate * maturity),  
           fwd = spot * exp((rate - div) * maturity),  
           std = vol * sqrt(maturity);  
    double d = log(fwd / strike) / std;  
    double d1 = d + 0.5 * std, d2 = d - 0.5 * std;  
    double p1 = normalCdf(d1), p2 = normalCdf(d2);  
    return df * (fwd * p1 - strike * p2);  
}
```



AAD

- In return, AAD is not an easy algorithm
 - Takes a bit of work to grasp
 - And a lot of work to implement correctly
- AAD works with computation graphs
 - Given a computation program
 - AAD builds the computation graph
 - In topological order by recording operations on a data structure called “tape”
 - And propagates derivatives (i.e. “adjoints”) back to front, using the chain rule

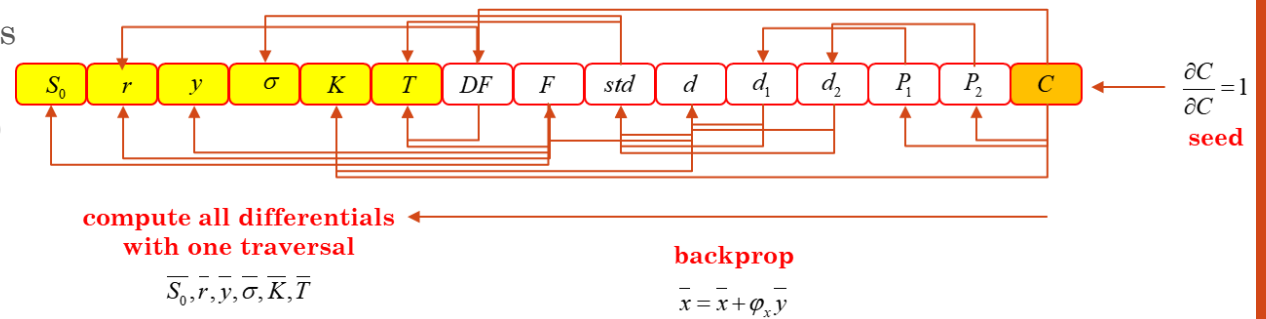
```
double blackScholes(
    const double spot,
    const double rate,
    const double div,
    const double vol,
    const double strike,
    const double maturity)
{
    double df = exp(-rate * maturity),
           fwd = spot * exp((rate - div) * maturity),
           std = vol * sqrt(maturity);
    double d = log(fwd / strike) / std;
    double d1 = d + 0.5 * std, d2 = d - 0.5 * std;
    double p1 = normalCdf(d1), p2 = normalCdf(d2);
    return df * (fwd * p1 - strike * p2);
}
```



AAD

- In return, AAD is not an easy algorithm
 - Takes a bit of work to grasp
 - And a lot of work to implement correctly
- AAD works with computation graphs
 - Given a computation program
 - AAD builds the computation graph
 - In topological order by recording operations on a data structure called “tape”
 - And propagates derivatives (i.e. “adjoints”) back to front, using the chain rule
 - Computing **all** derivatives with only **one** graph traversal

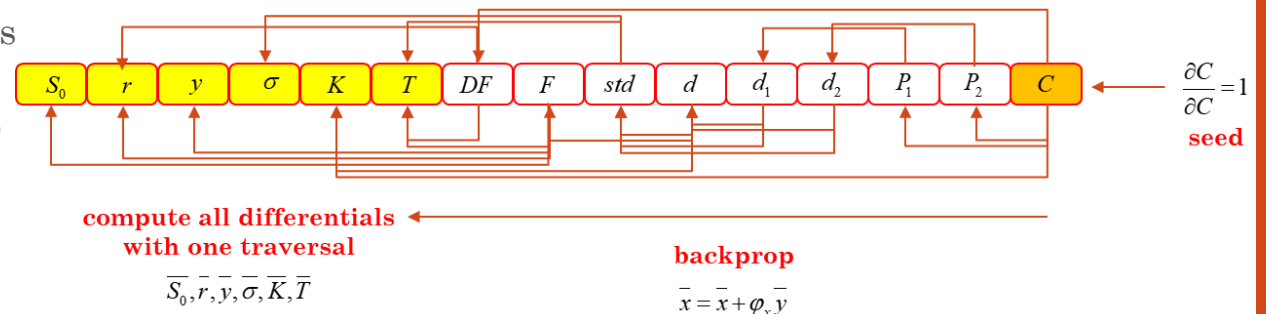
```
double blackScholes(
    const double spot,
    const double rate,
    const double div,
    const double vol,
    const double strike,
    const double maturity)
{
    double df = exp(-rate * maturity),
           fwd = spot * exp((rate - div) * maturity),
           std = vol * sqrt(maturity);
    double d = log(fwd / strike) / std;
    double d1 = d + 0.5 * std, d2 = d - 0.5 * std;
    double p1 = normalCdf(d1), p2 = normalCdf(d2);
    return df * (fwd * p1 - strike * p2);
}
```



AAD

- In return, AAD is not an easy algorithm
 - Takes a bit of work to grasp
 - And a lot of work to implement correctly
- AAD works with computation graphs
 - Given a computation program
 - AAD builds the computation graph
 - In topological order by recording operations on a data structure called “tape”
 - And propagates derivatives (i.e. “adjoints”) back to front, using the chain rule
 - Computing **all** derivatives with only **one** graph traversal
- See Modern Computational Finance (Wiley, 2018) for comprehensive presentation and implementation code

```
double blackScholes(
    const double spot,
    const double rate,
    const double div,
    const double vol,
    const double strike,
    const double maturity)
{
    double df = exp(-rate * maturity),
           fwd = spot * exp((rate - div) * maturity),
           std = vol * sqrt(maturity);
    double d = log(fwd / strike) / std;
    double d1 = d + 0.5 * std, d2 = d - 0.5 * std;
    double p1 = normalCdf(d1), p2 = normalCdf(d2);
    return df * (fwd * p1 - strike * p2);
}
```



source:

AAD & Backpropagation in Machine Learning and Finance,
Explained in 15min

(Savine, nov2019)

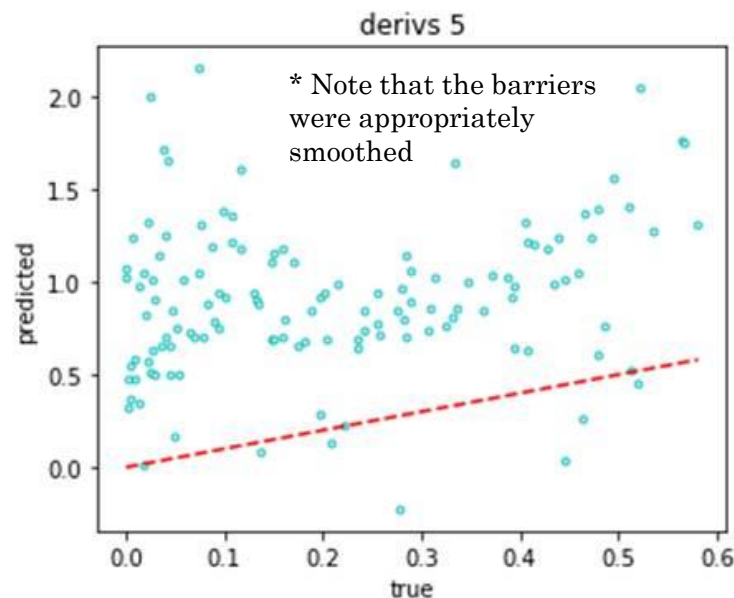
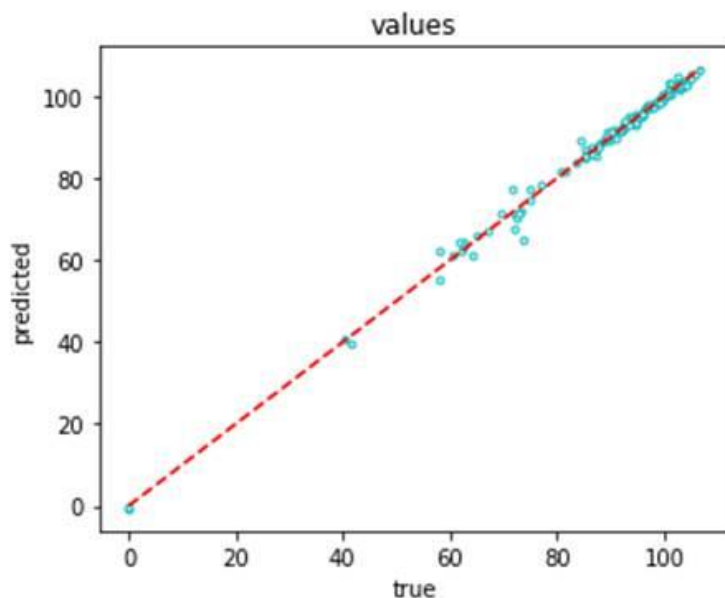
<https://www.youtube.com/watch?v=IcQkwwgPwfm4>

Conventional ML: Learning pricing functions

- LSM dataset $\{X^{(i)}, Y^{(i)} = h(X^{(i)}, \omega^{(i)})\}$
- Learn pricing function $v(x) = E[Y|X = x]$
- Conditional expectation = projection in L2 $E[Y|X] = \arg \min_{\text{measurables functions } \alpha(X)} MSE = E[(Y - \alpha(X))^2]$
- Strategy: use a universal approximator $\alpha(x; \mathcal{G}) \approx v(x) = E[Y|X = x]$
- Train by minimization of (empirical) MSE $\mathcal{G}^* = \arg \min_{\mathcal{G}} \left\{ E[(Y - \alpha(X; \mathcal{G}))^2] \approx \frac{1}{m} \sum_{i=1}^m (Y^{(i)} - \alpha(X^{(i)}; \mathcal{G}))^2 \right\}$
- $\alpha(x; \mathcal{G}^*)$ converges to $v(x) = E[Y|X = x]$ as the size m and capacity $\dim(\mathcal{G})$ grow to infinity

Example: best-of-four autocallable

- Best-of-four autocallable in Danske Bank's Superfly
- 4 correlated local volatility models a la Dupire
- Low dimension 4 but complex model and instrument
- 32k LSM training examples → decent approximation of values but deltas are completely off

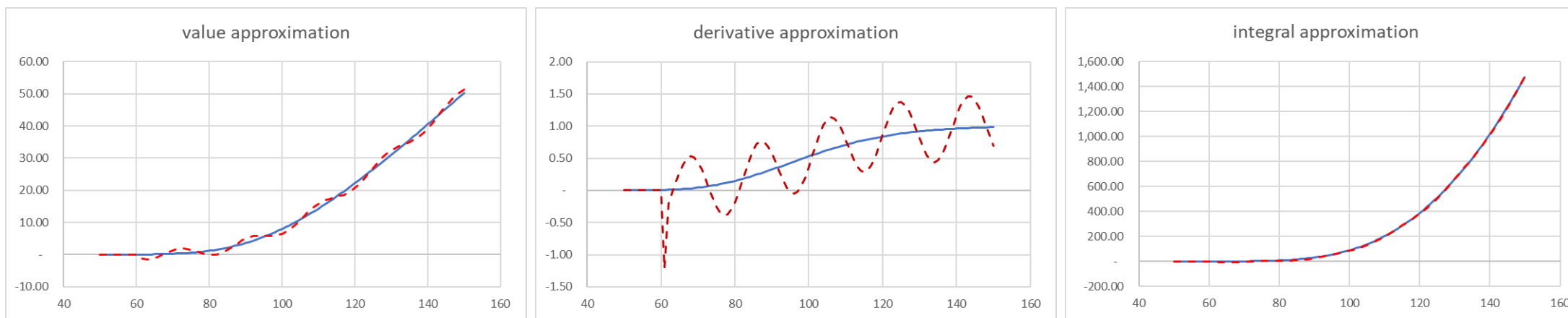


- Performance on independent test set
- Horizontal axis: correct values and deltas by nested MC
- Vertical axis: predictions of the trained ANN
- Error = distance to diagonal

Learning risk sensitivities

- Well-known in the field of function approximation:

the derivative of a good approximation is not a good approximation of the derivative



- Conversely:
the **integral** of a good (unbiased) approximation **is** a good approximation of the integral
- Suggests to learn good risk sensitivities (and good prices by integration)
rather than good prices (and poor sensitivities by differentiation)

Differential ML: Learning risk functions

- Differential dataset $\left\{X^{(i)}, Y^{(i)}, Z^{(i)} = \frac{\partial Y^{(i)}}{\partial X^{(i)}}\right\}$

- Pricing function $v(X) = E[Y|X]$

- Risk function by differentiation wrt X under E (assuming appropriate smoothing):

$$\frac{\partial v}{\partial X}(X) = E\left[\frac{\partial Y}{\partial X} | X\right] = E[Z | X] = \min_{\text{measurable functions } \frac{\partial \alpha}{\partial X}(X)} \overline{MSE} = E\left[\left|Z - \frac{\partial \alpha}{\partial X}(X)\right|^2\right]$$

- Train risk approximator $\frac{\partial \alpha}{\partial X}$ by minimization of differential MSE
 - MSE between predicted sensitivities $\frac{\partial \alpha}{\partial X}(X)$ and differential labels Z
 - Converges to correct risk sensitivities and gives us the correct pricing function alpha (modulo additive constant)

Have the cake and eat it too

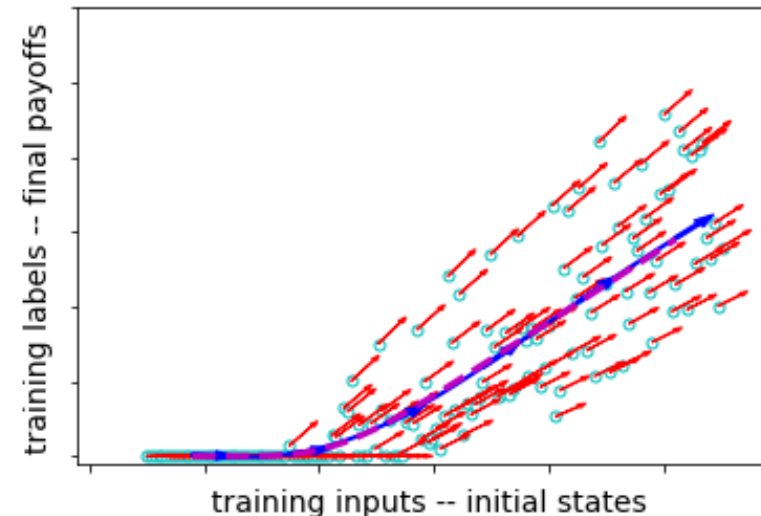
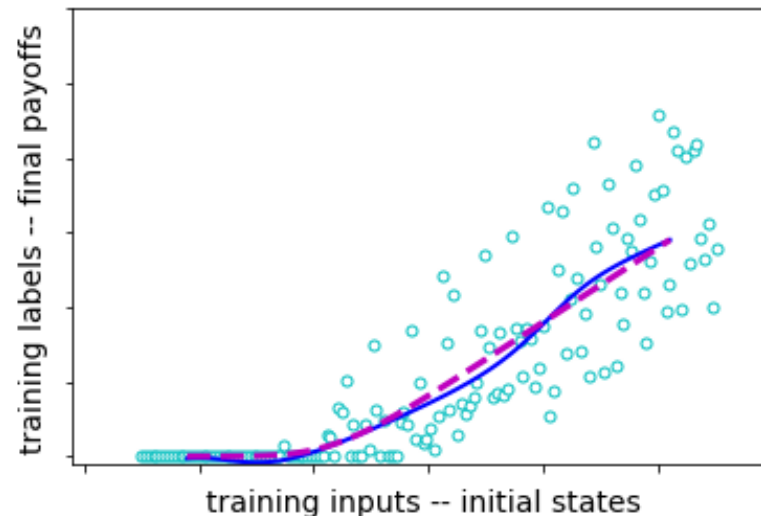
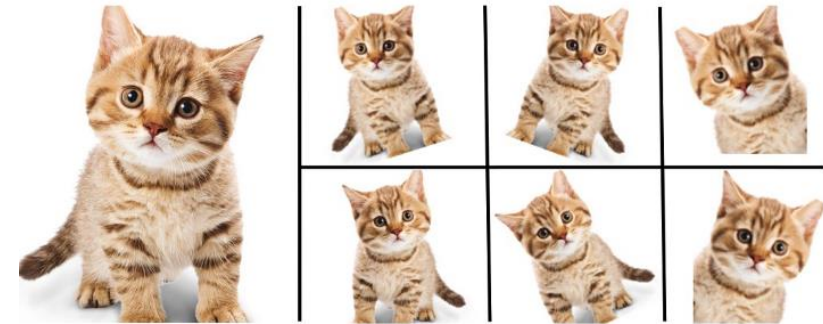
- In practice, it is best minimize the combined MSE: $\mathcal{G}^* = \arg \min \left\{ \text{MSE}(\mathcal{G}) + \sum_{j=1}^n \lambda_j \overline{\text{MSE}}_j(\mathcal{G}) \right\}$
 - Where $\overline{\text{MSE}}_j(\mathcal{G}) = E \left[\left(Z_j - \frac{\partial \alpha}{\partial X_j}(X; \mathcal{G}) \right)^2 \right] \approx \frac{1}{m} \sum_{i=1}^m \left(Z_j^{(i)} - \frac{\partial \alpha}{\partial X_j}(X^{(i)}; \mathcal{G}) \right)^2$ is the (mean-squared) error on derivative wrt j-th input X_j
 - And $\lambda_j = \frac{E[Y^2]}{E[Z_j^2]}$ are normalizing weights – not tunable hyperparameters

Differential regularization

- Recall optimization problem statement: $\mathcal{G}^* = \arg \min \left\{ MSE(\mathcal{G}) + \sum_{j=1}^n \lambda_j \overline{MSE}_j(\mathcal{G}) \right\}$
- It is of the general form: $\mathcal{G}^* = \arg \min \{ MSE(\mathcal{G}) + \lambda \cdot \text{penalty} \}$ i.e. a regularization
- With classic regularization (ridge, lasso, ...) the penalty is somewhat arbitrary:
 - Magnitude of coefficients theta or sometimes magnitude of first or second-order derivatives
 - Prevents the optimizer from overfitting noisy data, thereby reducing variance
 - But introduces a bias
 - A sweet spot must be found for the regularization parameter lambda, i.e. bias-variance trade-off
 - Generally optimized by cross-validation
- Differential regression penalises wrong derivatives
 - Also reduces overfitting and variance
 - Without introducing any bias – since correct derivatives give correct value by integration
 - No bias-variance trade-off, no hyperparameter that needs tuning, no cross-validation

Differential augmentation

- Additional data: for each example X , we have one label Y and n additional differential labels Z
- Similar to data augmentation in computer vision
 - New data by rotation, zooming or cropping existing pictures
 - Teaches invariances: a rotated cat is still a cat
 - Augmented data often performs better than new data
- Similarly, differential data Z
 - Cheaply augments the dataset (with AAD)
 - Teaches the **shape** of the function -- DML performs better with small datasets than ML with larger datasets

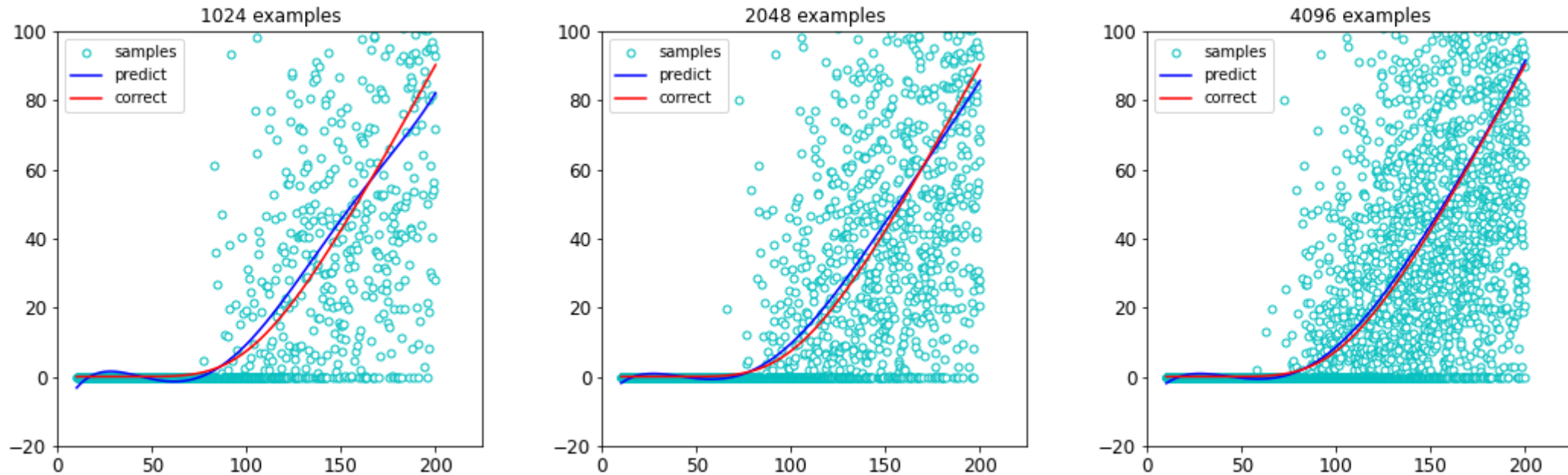


Basis function regression

- Pick K basis functions $\varphi_1, \dots, \varphi_K$ of $x \in \mathbb{R}^n$, ex: monomials of degree up to p $\prod_{j=1}^n x_j^{p_j}, \sum_{j=1}^n p_j \leq p$
- Train universal approximator $\alpha(x; \mathcal{g}) = \sum_{k=1}^K g_k \varphi_k(x)$
- By minimisation of MSE $MSE(\mathcal{g}) = E[(Y - \alpha(X; \mathcal{g}))^2]$
- Analytic solution $\mathcal{g}^* = \arg \min MSE(\mathcal{g}) = C_{\varphi\varphi}^{-1} C_{\varphi y}$ “normal equation”
- Where covariances $C_{\varphi\varphi} = E[\varphi(X)\varphi(X)^T] \in \mathbb{R}^{K \times K}$ and $C_{\varphi y} = E[\varphi(X)Y] \in \mathbb{R}^K$ are estimated over training data

Example: Black and Scholes

European call in Black & Scholes : degree 5 polynomial regression vs correct formula



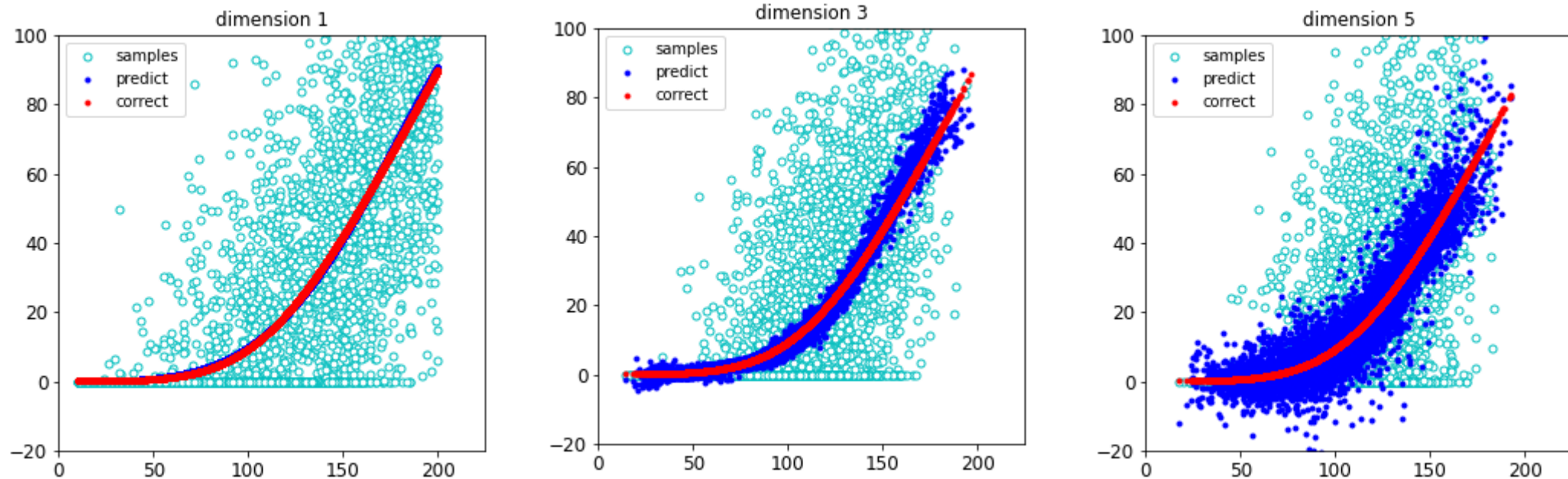
source: <https://github.com/differential-machine-learning/notebooks/blob/master/DifferentialRegression.ipynb>

Example: Gaussian basket

- Black & Scholes is **not** a representative example
- Because its dimension is 1: $X = \{\text{spot}\}$
- Most interesting situations in finance are high-dimensional
- A more representative example is the basket option in the correlated Gaussian model
 - Simple example but with multidimensional state $X = \{\text{spot}[1], \text{spot}[2], \dots, \text{spot}[n]\}$
 - Cashflow: $h(X; \omega) = (a^T S_T - K)^+ = \left(a^T \left[X + \sqrt{\Sigma} \sqrt{T} N^{-1}(\omega) \right] - K \right)^+$ with $a \in \mathbb{R}^n$: fixed basket weights
 $\Sigma \in \mathbb{R}^{n \times n}$: covariance matrix
 - Analytic pricing $v(x) = \text{Bachelier}(a^T x, \sqrt{a^T \Sigma a}; K, T)$
 - Single-dimensional problem in disguise $h(X; \omega) = (a^T X + a^T \sqrt{\Sigma} \sqrt{T} N^{-1}(\omega) - K)^+ = f(a^T X; \omega)$
 - Can the machine make sense of that?

Example: Gaussian basket

correlated Gaussian basket : polynomial deg 5 regression vs formula, 4096 training examples, independent test set



source: <https://github.com/differential-machine-learning/notebooks/blob/master/DifferentialRegression.ipynb>

Curse of dimensionality

- Polynomial regression degree p : basis functions are K monomials of the form $\prod_{j=1}^n x_j^{p_j}, \sum_{j=1}^n p_j \leq p$
- Also K learnable parameters $\theta_1, \dots, \theta_K$
- Simple combinatorics: $K = \frac{(n+p)!}{n!p!}$ grows like n^p
- With that many free coefficients, regression **overfits** the noise of the training set and ends up **interpolating** the noisy data
- Often called ‘curse of dimensionality’
- Affects **all** flavours of basis function regression, not only polynomial

Overcoming the curse

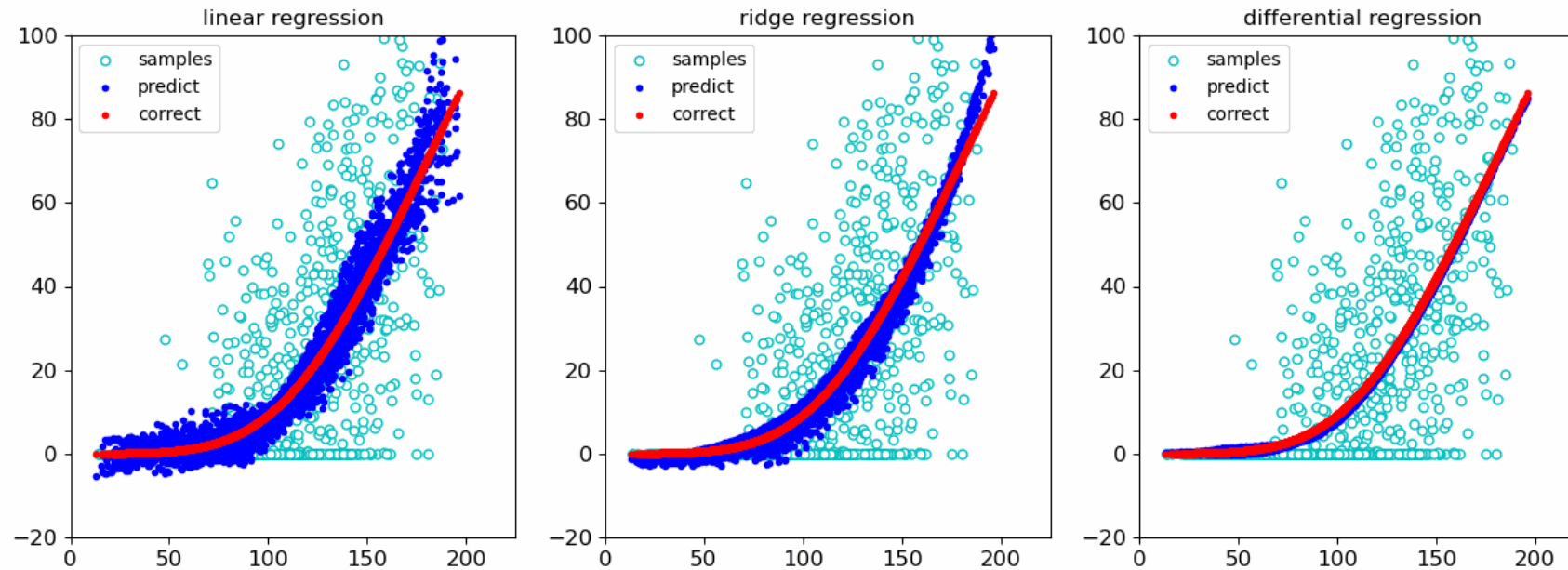
- Regularization techniques: ridge, lasso...
 - Moderately effective
- Manual feature engineering
 - Reduce dimension with human analysis and prior knowledge of pricing model and Derivative transaction
 - Example 1: basket option under Gaussian assumption $\rightarrow$ we know that $v(x) = g(a^T x)$
 - Example 2: Bermudan option under LMM $\rightarrow v(x) \approx g(\text{short_rate}(x), \text{long_rate}(x))$ (see Andreasen + Andersen, 2000)
 - Applied in most current implementations of LSM
 - Expensive, requires maintenance, prone to error, does not scale to arbitrary trading books
- Modern ML offers **automatic** feature engineering, e.g. with artificial neural networks (ANN)

Differential regression

- Basis function regression: $\alpha(x; \mathcal{g}) = \sum_{k=1}^K g_k \varphi_k(x)$
- Sensitivity prediction: $\frac{\partial \alpha}{\partial X_j}(X^{(i)}; \mathcal{g}) = \sum_{k=1}^K g_k \frac{\partial \varphi_k}{\partial X_j}(X^{(i)})$
- Training by minimization of combined MSE: $\mathcal{g}^* = \arg \min \left\{ \text{MSE}(\mathcal{g}) + \sum_{j=1}^n \lambda_j \overline{\text{MSE}}_j(\mathcal{g}) \right\}$ $\overline{\text{MSE}}_j(\mathcal{g}) = E \left[\left(Z_j - \frac{\partial \alpha}{\partial X_j}(X; \mathcal{g}) \right)^2 \right]$
- Analytic solution: modified normal equation $\mathcal{g}^* = \left(C_{\varphi\varphi} + \sum_{j=1}^n \lambda_j C_{jj}^{\varphi} \right)^{-1} \left(C_{\varphi y} + \sum_{j=1}^n \lambda_j C_j^{\varphi z} \right)$ (solve for 0 gradient)
- Where $C_{jj}^{\varphi} = E \left[\frac{\partial \varphi}{\partial X_j}(X) \frac{\partial \varphi}{\partial X_j}(X)^T \right] \in \mathbb{R}^{K \times K}$ and $C_j^{\varphi z} = E \left[\frac{\partial \varphi}{\partial X_j}(X) Z_j \right] \in \mathbb{R}^K$
- See details and implementation on github.com/differential-machine-learning/notebooks/blob/master/DifferentialRegression.ipynb

Diff regression: performance

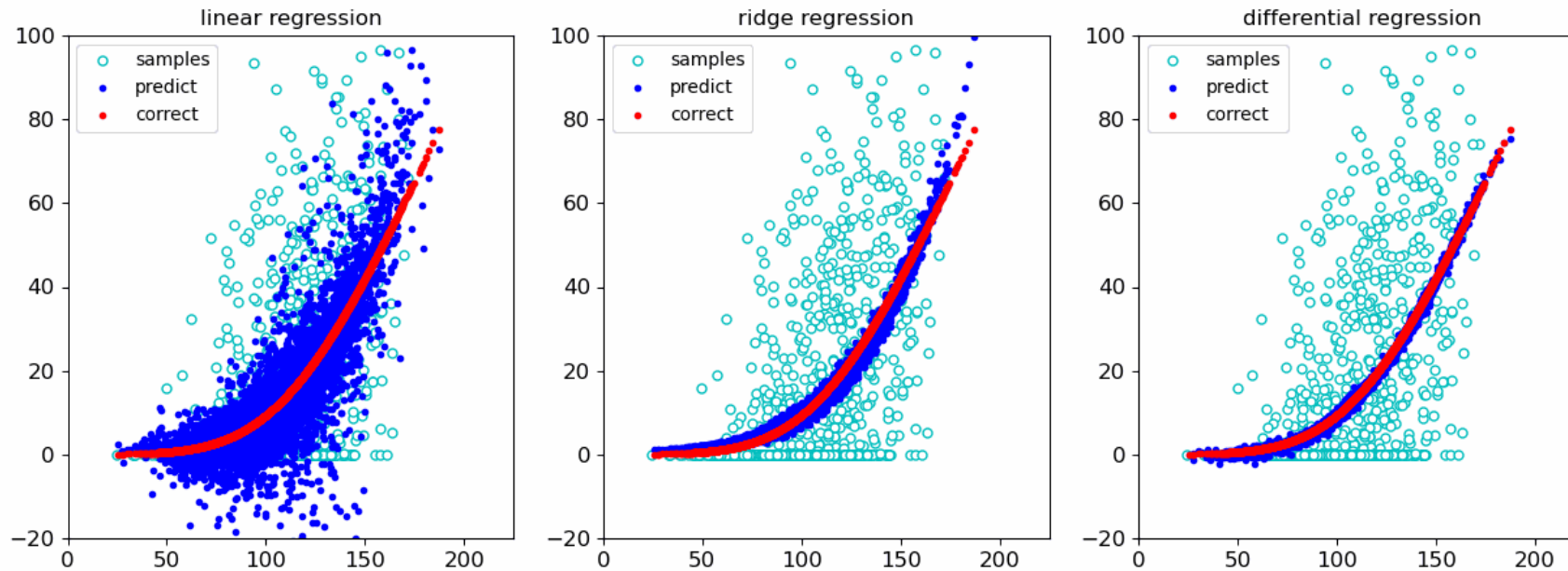
correlated Gaussian basket **dim 3** : deg 5 regression, only 1024 training examples, ridge optimized by CV



source: <https://github.com/differential-machine-learning/notebooks/blob/master/DifferentialRegression.ipynb>

Diff regression: performance

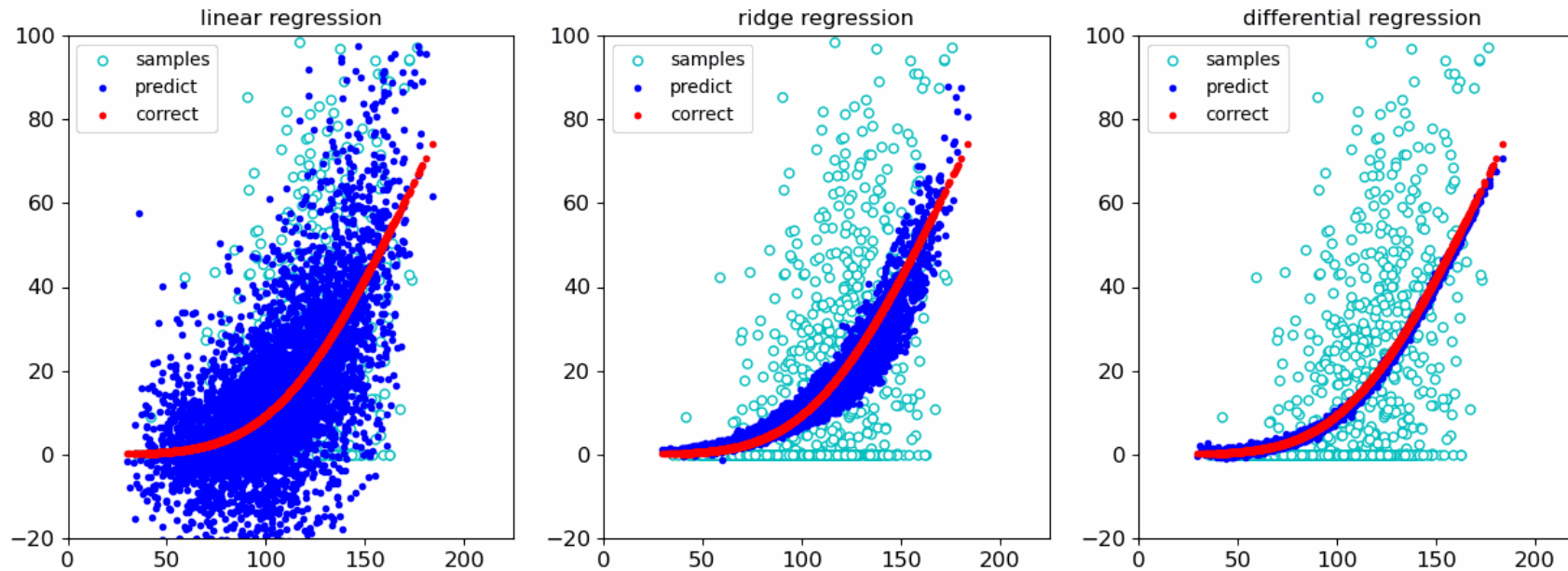
correlated Gaussian basket **dim 5** : deg 5 regression, only 1024 training examples, ridge optimized by CV



source: <https://github.com/differential-machine-learning/notebooks/blob/master/DifferentialRegression.ipynb>

Diff regression: performance

correlated Gaussian basket **dim 7** : deg 5 regression, only 1024 training examples, ridge optimized by CV

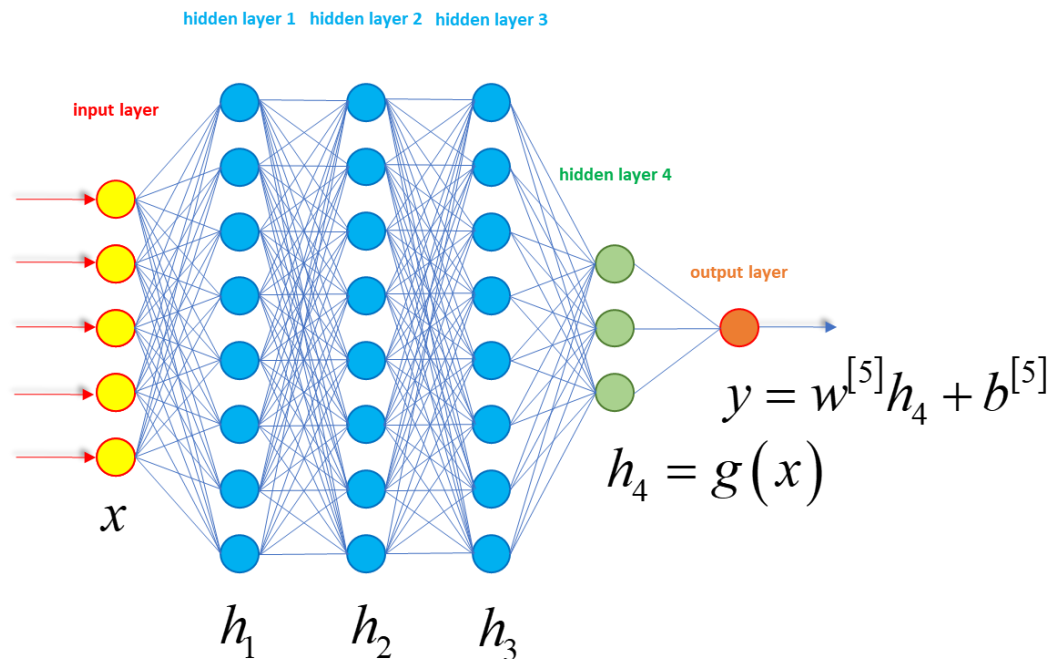


source: <https://github.com/differential-machine-learning/notebooks/blob/master/DifferentialRegression.ipynb>

Differential regression

- Sharp improvement over classic regression, with or without regularization
- Not a silver bullet: still vulnerable to the curse of dimensionality
- To overcome it, we need more sophisticated ML models designed for that purpose
- Like artificial neural networks (ANN)

Artificial neural networks



g encoded in the hidden connection weights

- ANN generalize basis function regression
 - output layer = linear regression
 - where regression functions are the units of h_4
 - which are functions of x encoded in the hidden layers
 - regression functions are not fixed
 - learned from data
 - we decide how many of them we want
 - the ANN learns the “best” ones by minimization of cost
 - number of connection weights grows only **linearly** in $n = \dim(x)$
- escape curse of dimensionality!

Feedforward Neural Networks

- Predict with feedforward equations:

$$x^{[0]} = x \in \mathbb{R}^n \quad \text{input layer}$$

$$x^{[l]} = W^{[l]} g^{[l-1]}(x^{[l-1]}) + b^{[l]} \in \mathbb{R}^{n_l} \quad \text{hidden layers}$$

$$\alpha = x^{[L]} \in \mathbb{R} \quad \text{output layer}$$

- Learnable parameters are connection weights and biases $\theta = \{(W^{[l]}, b^{[l]}), 1 \leq l \leq L\}$

- Trained by minimization of MSE with (variants of) gradient descent $\theta_{i+1} = \theta_i - \lambda \frac{\partial \text{MSE}(\theta_i)}{\partial \theta_i}$

- Gradients dMSE / dTheta are computed by backpropagation:

$$\bar{\alpha} = \overline{x^{[L]}} = \frac{\partial \text{MSE}}{\partial v}$$

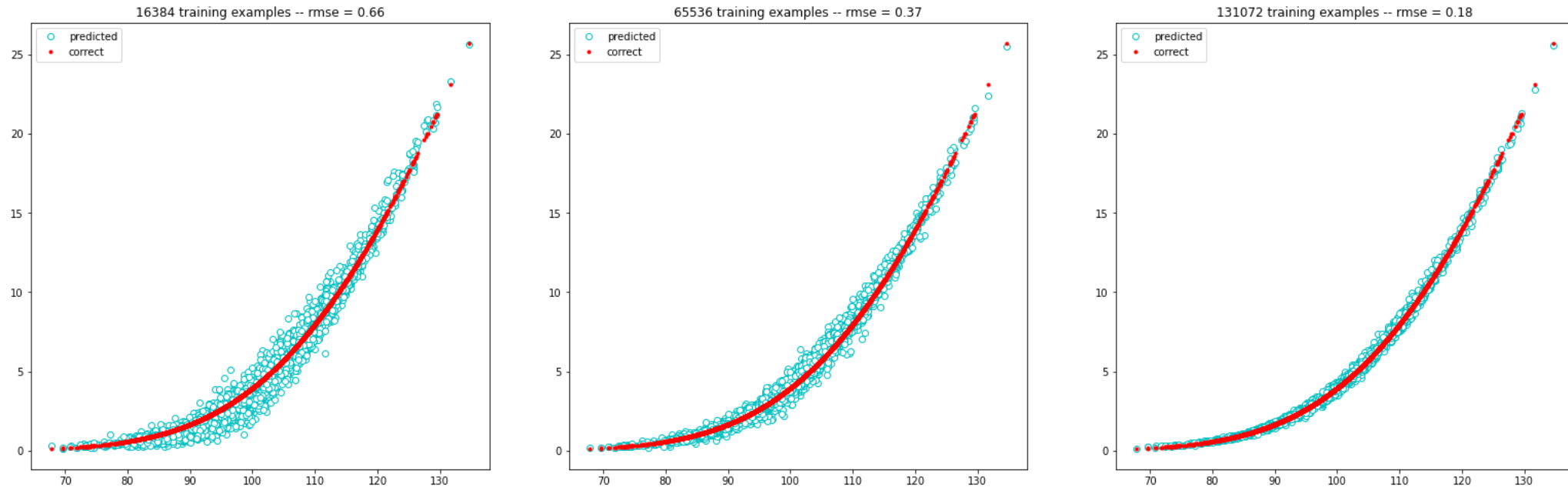
$$\overline{x^{[l-1]}} = g^{[l-1]'}(x^{[l-1]}) \otimes W^{[l]T} \bar{x}^{[l]}$$

$$\frac{\partial \text{MSE}}{\partial W^{[l]}} = \overline{x^{[l]}} g^{[l-1]}(x^{[l-1]})^T \quad \text{and} \quad \frac{\partial \text{MSE}}{\partial b^{[l]}} = \overline{x^{[l]}}$$

using the “bar” notation $\overline{\text{thing}} = \frac{\partial \text{MSE}}{\partial \text{thing}}$

ANN: pricing performance

correlated Gaussian basket **dim 30** : ANN vs formula on independent test set



source: <https://github.com/differential-machine-learning/notebooks/blob/master/DifferentialMLTF2.ipynb>

Differential ANN

- Training by minimization of combined MSE: $\mathcal{G}^* = \arg \min \left\{ \text{MSE}(\mathcal{G}) + \sum_{j=1}^n \lambda_j \overline{\text{MSE}}_j(\mathcal{G}) \right\}$

$$\mathbf{x}^{[0]} = \mathbf{x} \in \mathbb{R}^n$$

- Predicts values α with feedforward equations: $\mathbf{x}^{[l]} = \mathbf{W}^{[l]} \mathbf{g}^{[l-1]}(\mathbf{x}^{[l-1]}) + \mathbf{b}^{[l]} \in \mathbb{R}^{n_l}$

$$\alpha = \mathbf{x}^{[L]} \in \mathbb{R}$$

- Efficiently predicts sensitivities $\bar{\mathbf{x}} = \frac{\partial \alpha}{\partial \mathbf{x}}$ with backpropagation:

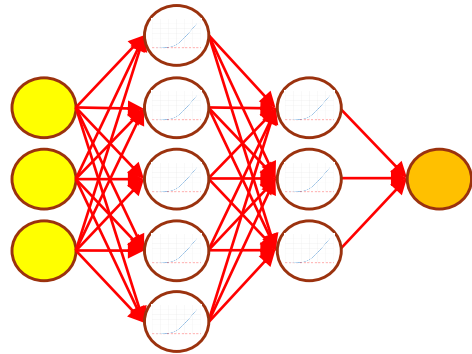
$$\begin{array}{ccc}
 \frac{\partial \text{MSE}}{\partial \mathcal{G}} & & \frac{\partial \alpha}{\partial \mathbf{x}} \\
 \bar{\alpha} = \overline{\mathbf{x}^{[L]}} = \frac{\partial \text{MSE}}{\partial v} & & \bar{\alpha} = 1 \\
 \overline{\mathbf{x}^{[l-1]}} = \mathbf{g}^{[l-1]'}(\mathbf{x}^{[l-1]}) \otimes \mathbf{W}^{[l]T} \overline{\mathbf{x}^{[l]}} & \longrightarrow & \overline{\mathbf{x}^{[l-1]}} = \mathbf{g}^{[l-1]'}(\mathbf{x}^{[l-1]}) \otimes \mathbf{W}^{[l]T} \overline{\mathbf{x}^{[l]}} \\
 \frac{\partial \text{MSE}}{\partial \mathbf{W}^{[l]}} = \overline{\mathbf{x}^{[l]}} \mathbf{g}^{[l-1]'}(\mathbf{x}^{[l-1]})^T \text{ and } \frac{\partial \text{MSE}}{\partial \mathbf{b}^{[l]}} = \overline{\mathbf{x}^{[l]}} & & \bar{\mathbf{x}} = \overline{\mathbf{x}^{[0]}}
 \end{array}$$

* Note that the bar notation now denotes derivatives of alpha

- Backpropagation defines a dual neural network!

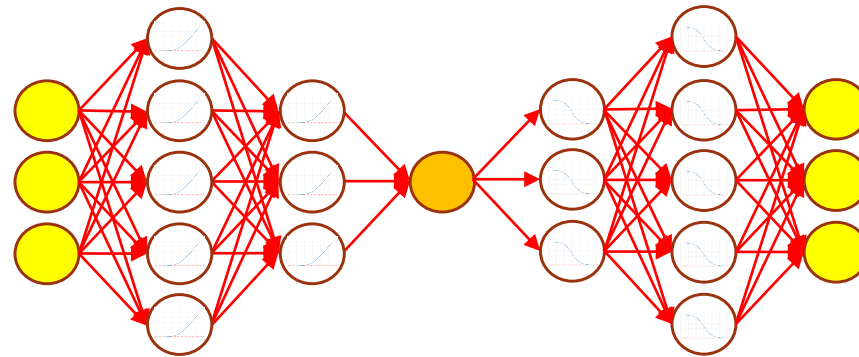
Twin ANN

Combine value and risk prediction with a twin network



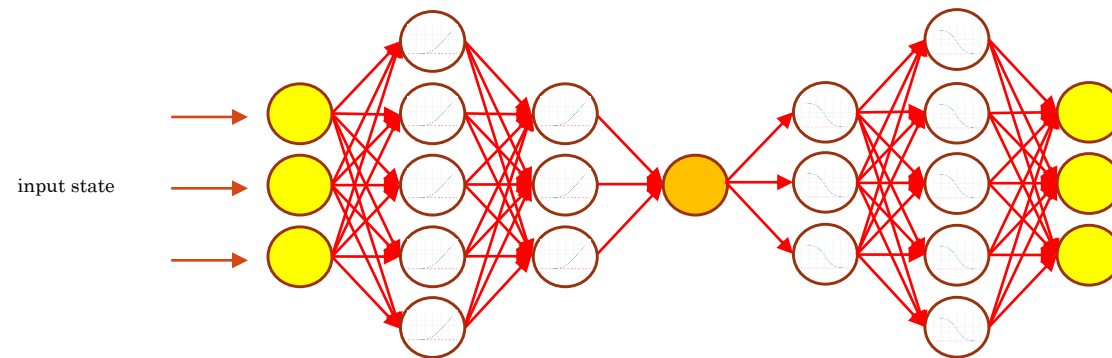
Twin ANN

Combine value and risk prediction with a twin network



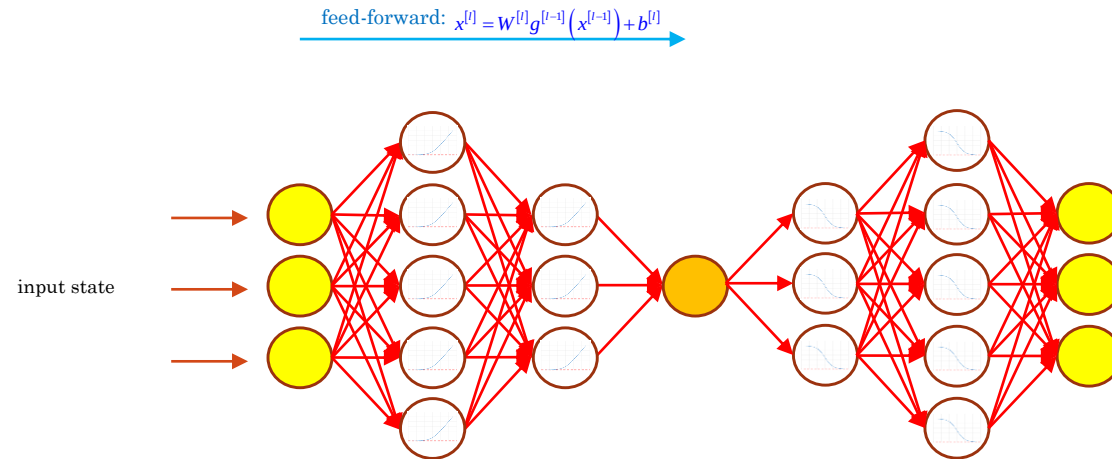
Twin ANN

Combine value and risk prediction with a twin network



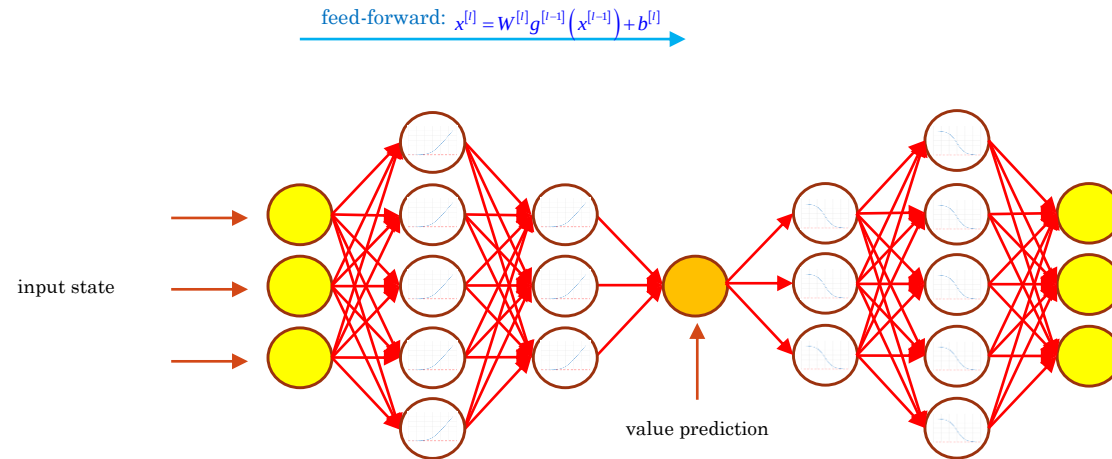
Twin ANN

Combine value and risk prediction with a twin network
Predict values by feedforward induction



Twin ANN

Combine value and risk prediction with a twin network
Predict values by feedforward induction

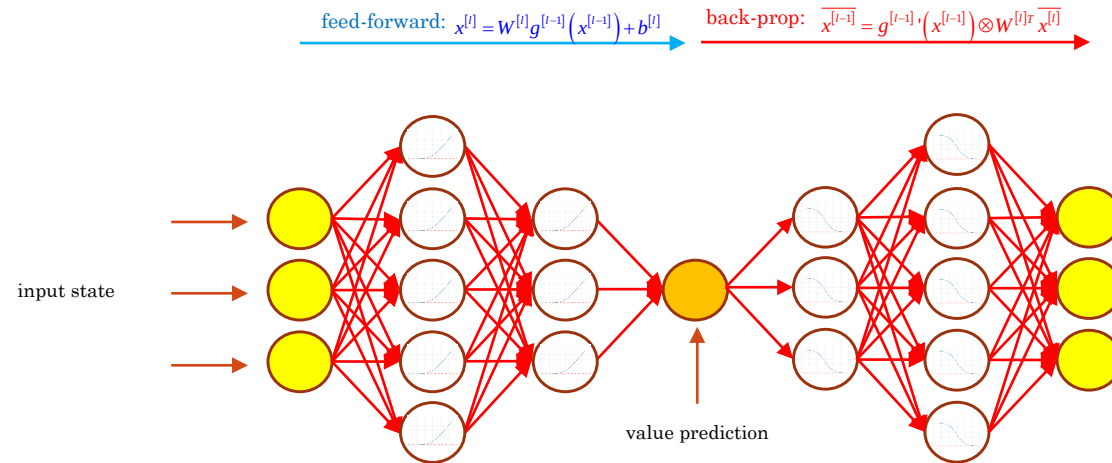


Twin ANN

Combine value and risk prediction with a twin network

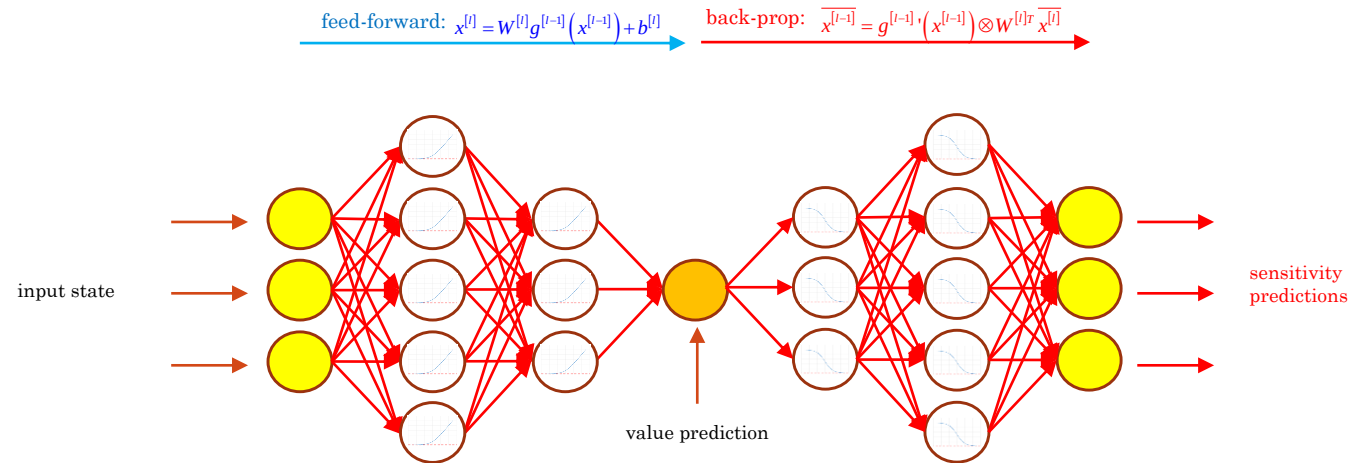
Predict values by feedforward induction

And then risks with backprop equations through the dual network



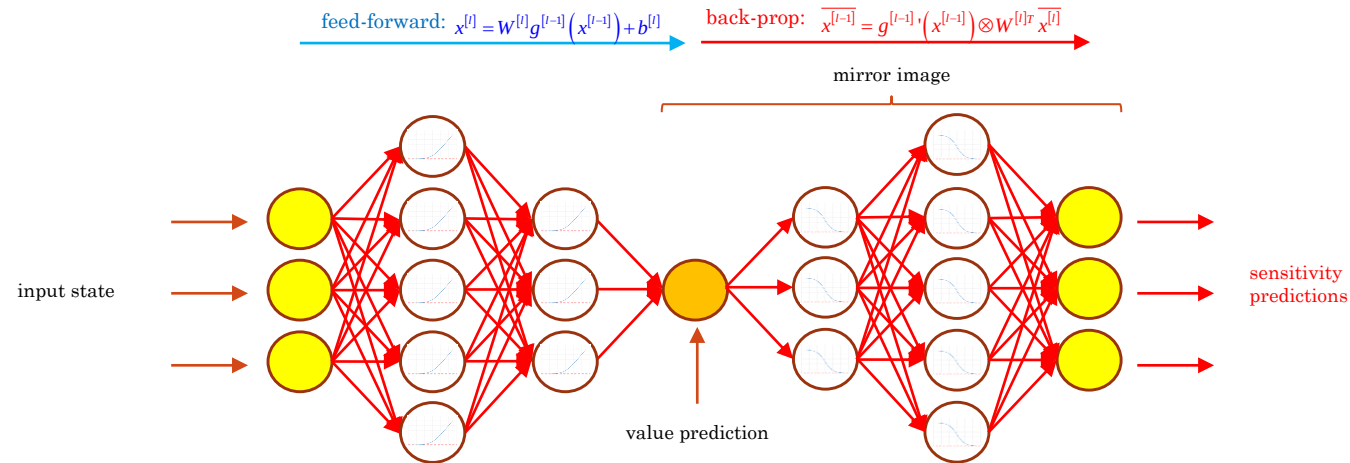
Twin ANN

Combine value and risk prediction with a twin network
Predict values by feedforward induction
And then risks with backprop equations through the dual network



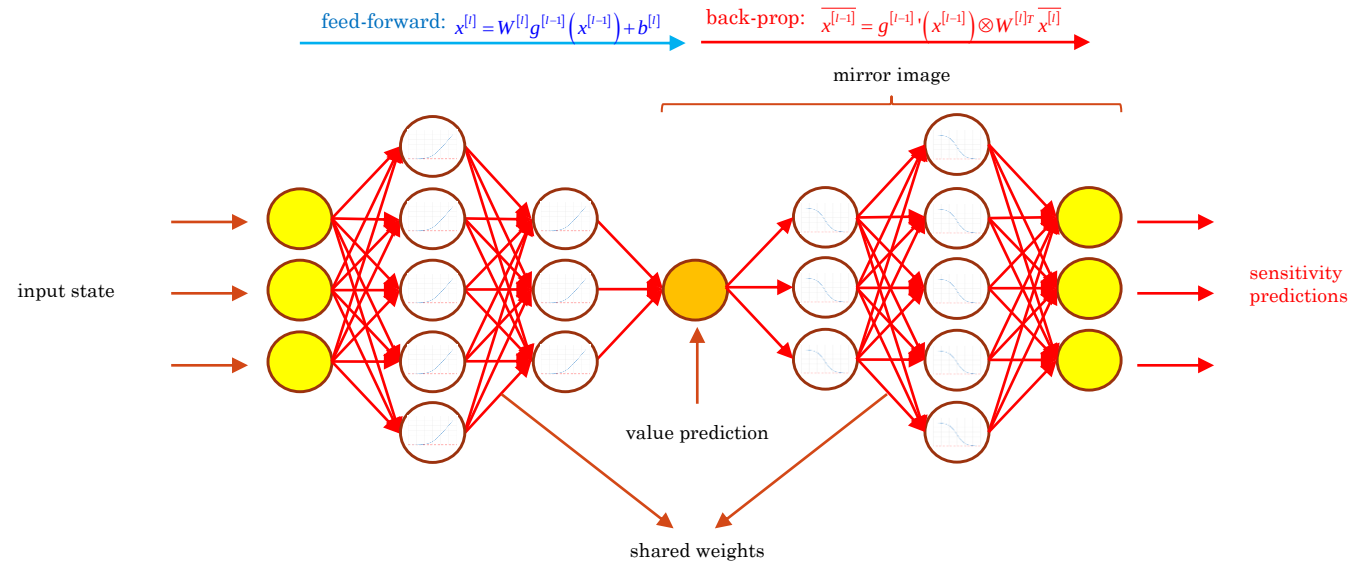
Twin ANN

Combine value and risk prediction with a twin network
Predict values by feedforward induction
And then risks with backprop equations through the dual network



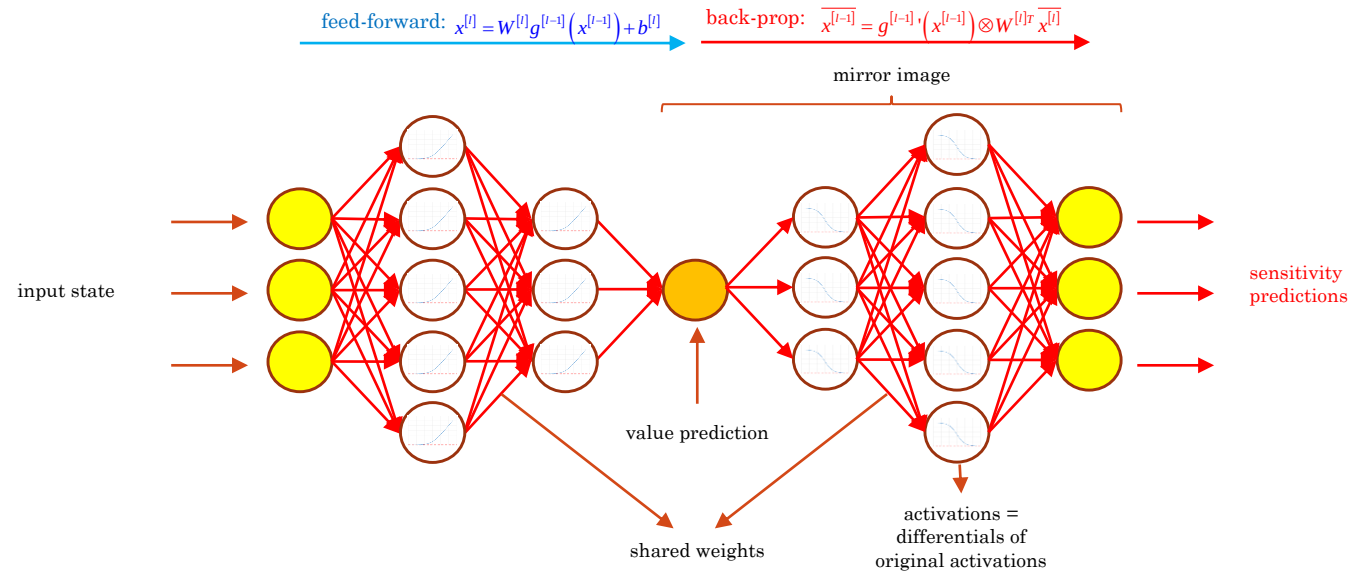
Twin ANN

Combine value and risk prediction with a twin network
Predict values by feedforward induction
And then risks with backprop equations through the dual network



Twin ANN

Combine value and risk prediction with a twin network
Predict values by feedforward induction
And then risks with backprop equations through the dual network



Twin ANN: implementation

- Twin ANN: predicts values and risks $\alpha(x; \mathcal{G})$ and $\frac{\partial \alpha}{\partial x}(x; \mathcal{G})$ with one forward pass
- Trained by minimization of combined MSE $\mathcal{G}^* = \arg \min \left\{ \sum_{i=1}^m \left[Y^{(i)} - \alpha(X^{(i)}; \mathcal{G}) \right]^2 + \sum_{j=1}^n \lambda_j \sum_{i=1}^m \left(Z_j^{(i)} - \frac{\partial \alpha}{\partial X_j}(X^{(i)}; \mathcal{G}) \right)^2 \right\}$
- With gradient descent and (another round of) backpropagation through the twin network (automated in TensorFlow)

Twin ANN: implementation

- Better understood with implementation code
- TensorFlow implementation available on GitHub (differential-machine-learning)

<https://github.com/differential-machine-learning/notebooks/blob/master/DifferentialMLTF2.ipynb>

- With applications and a discussion of implementation details
- Run it on your browser (or iPhone) **on Google's GPU servers:**

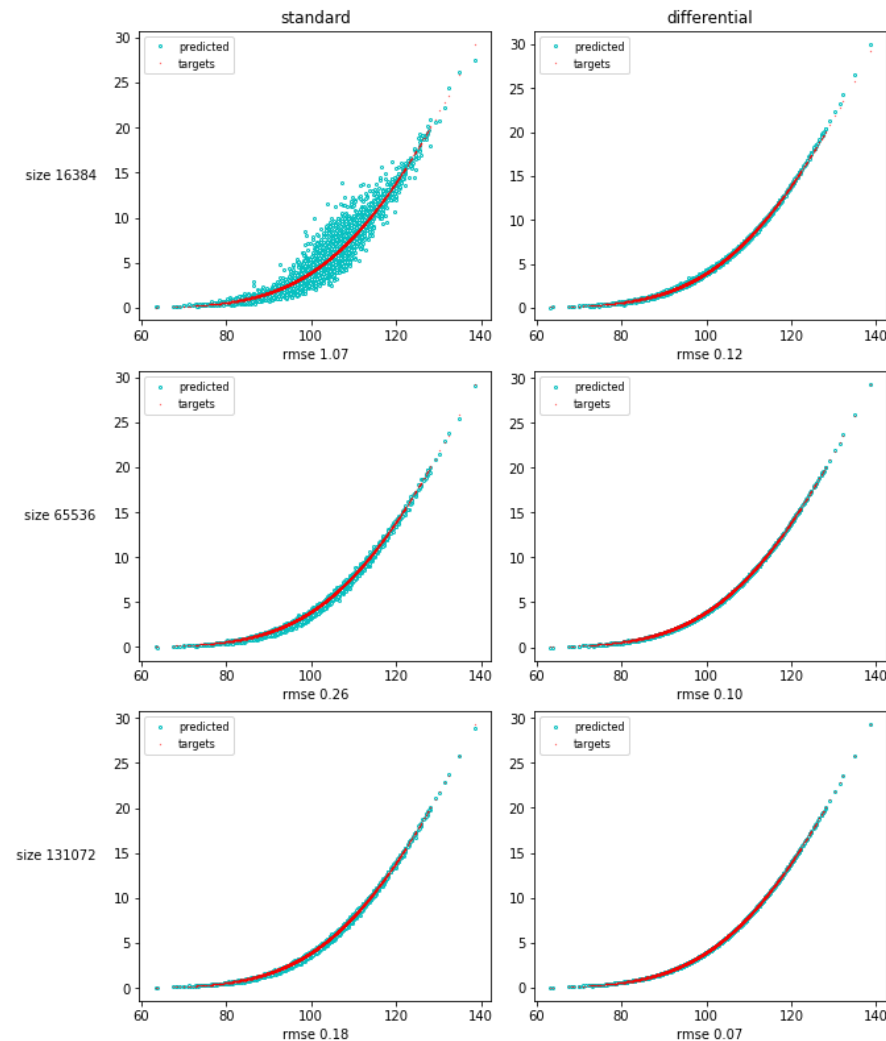


ANN: pricing performance

correlated Gaussian basket **dim 30** :
ANN vs DANN vs formula on independent test set

source:

<https://github.com/differential-machine-learning/notebooks/blob/master/DifferentialMLTF2.ipynb>

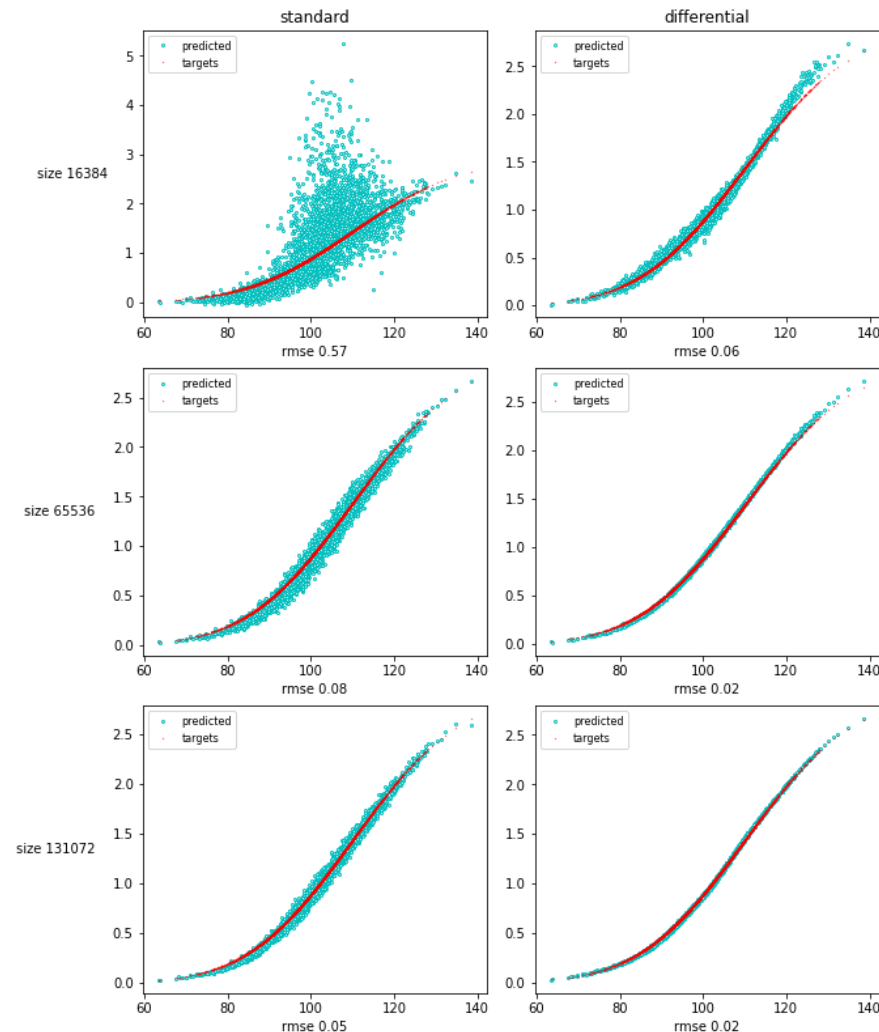


ANN: risk performance

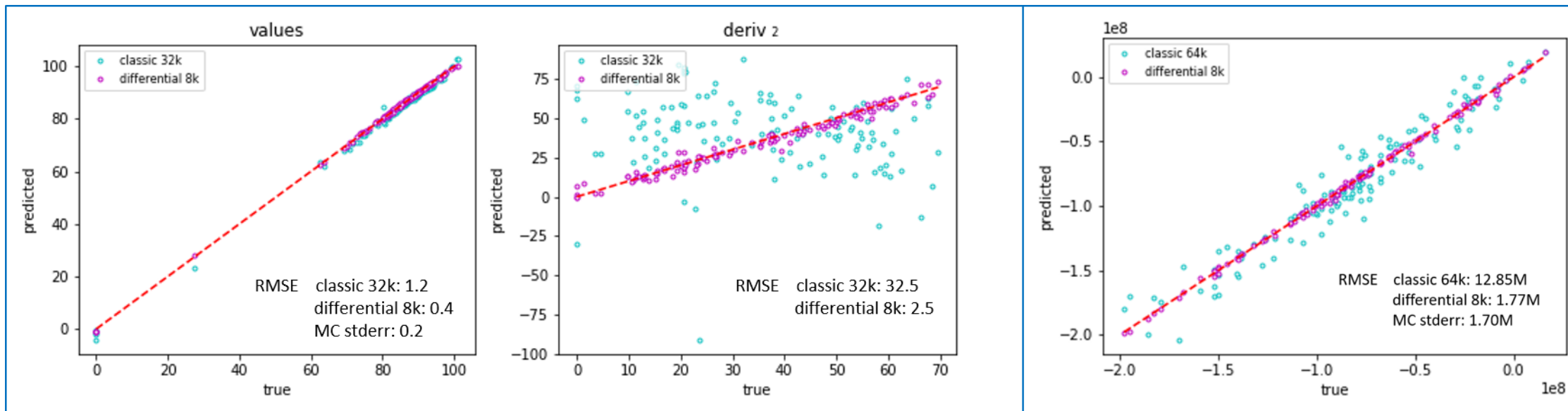
delta wrt 1st asset :
ANN vs DANN vs formula on independent test set

source:

<https://github.com/differential-machine-learning/notebooks/blob/master/DifferentialMLTF2.ipynb>



DANN: real-world performance



worst-of-four autocallable

real-world netting set

classical network predicts decent values with 32k examples but random deltas

twin network predicts almost perfect values with only 8k examples and solid deltas

classical network overfits with 64k examples due to high dimensionality, it does converge but needs a much bigger dataset

twin network converges almost perfectly with only 8k examples

source:

Risk.net: <https://www.risk.net/cutting-edge/banking/7688441/differential-machine-learning-the-shape-of-things-to-come>

Beyond pricing functions

- Differential ML has many applications besides learning pricing functions
- Besides, we haven't completely resolved the basket option problem
 - Under Gaussian assumption (and approximately otherwise): one-dimensional problem in disguise

$$h(X; \omega) = f(a^T X; \omega)$$

- A “good” algorithm should:
 - Identify this from data
 - Reliably reduce dimension to one, by projection on the appropriate axis
 - Enable effective basis function regression, irrespective of original dimension
 - Without ANN or other heavyweight processing

Differential PCA

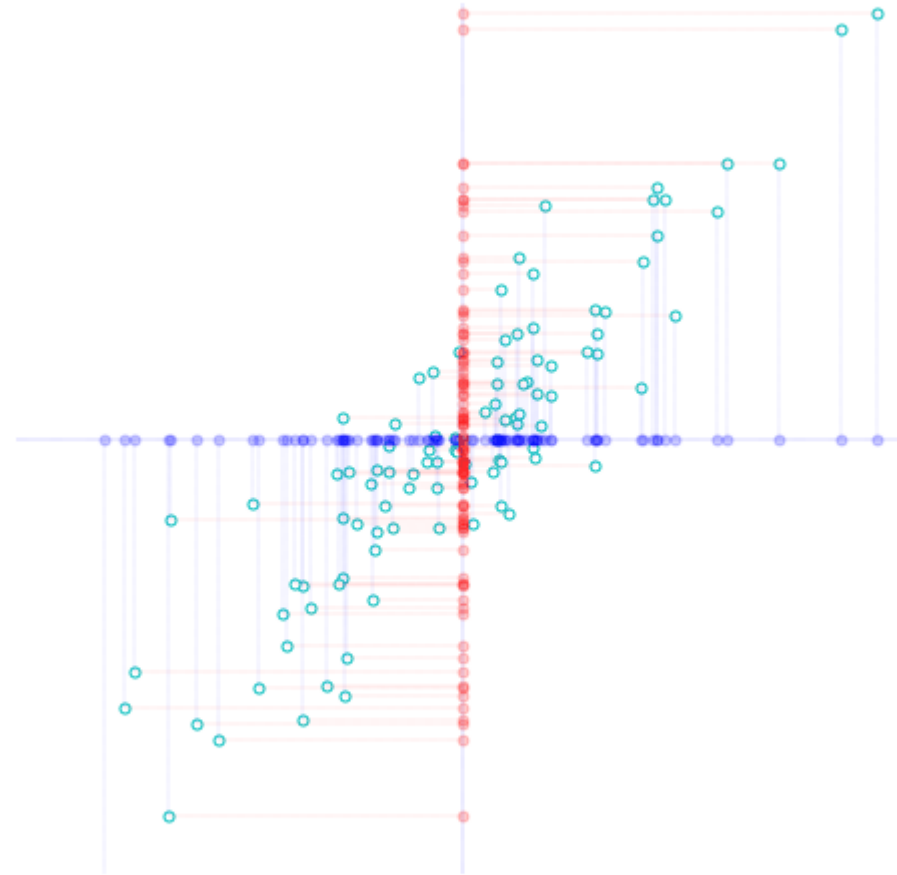
- One differential ML algorithm does exactly this: differential PCA
- Differential PCA = eigenvalue decomposition of the covariance of differential labels

$$C_{zz} = E[ZZ^T] = PDP^T$$

- Eigenvectors P: axes of orthogonal directional sensitivities
- Eigenvalues D: measure magnitude of directional sensitivities
- Axes with immaterial sensitivities are irrelevant for the problem and safely truncated
- Published in Sep21 Risk Magazine: <https://www.risk.net/cutting-edge/banking/7877231/axes-that-matter-pca-with-a-difference>
- Implementation code: <https://github.com/differential-machine-learning/notebooks/blob/master/DifferentialPCA.ipynb>

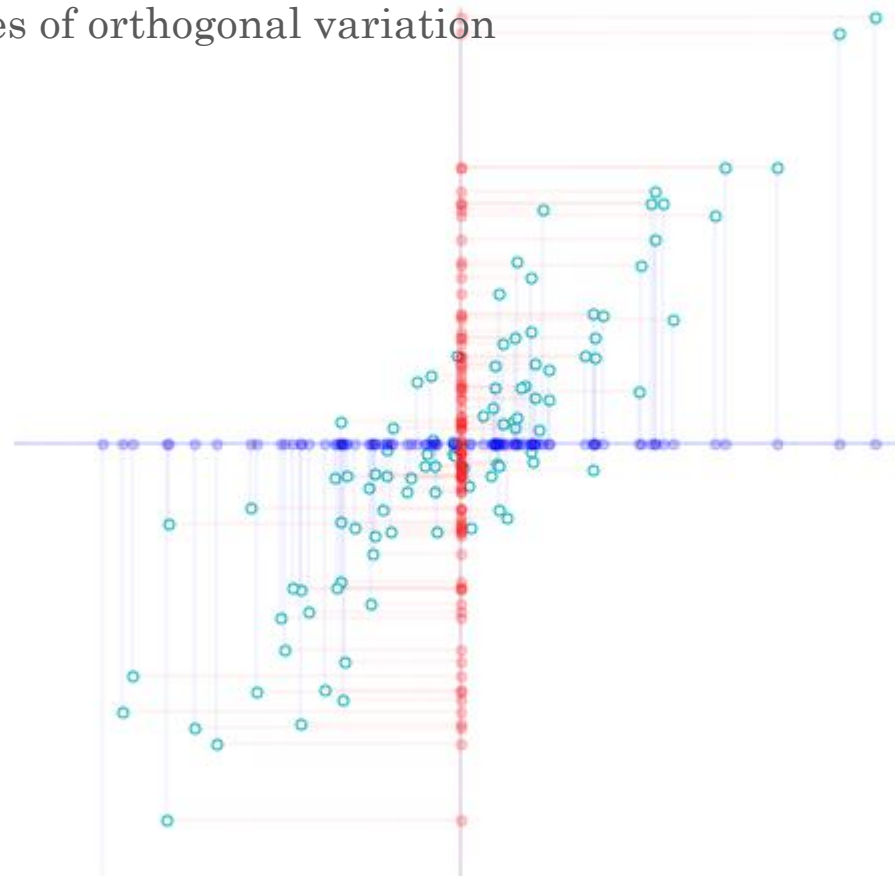
DPCA: intuition

- Two correlated asset prices X_1 and X_2



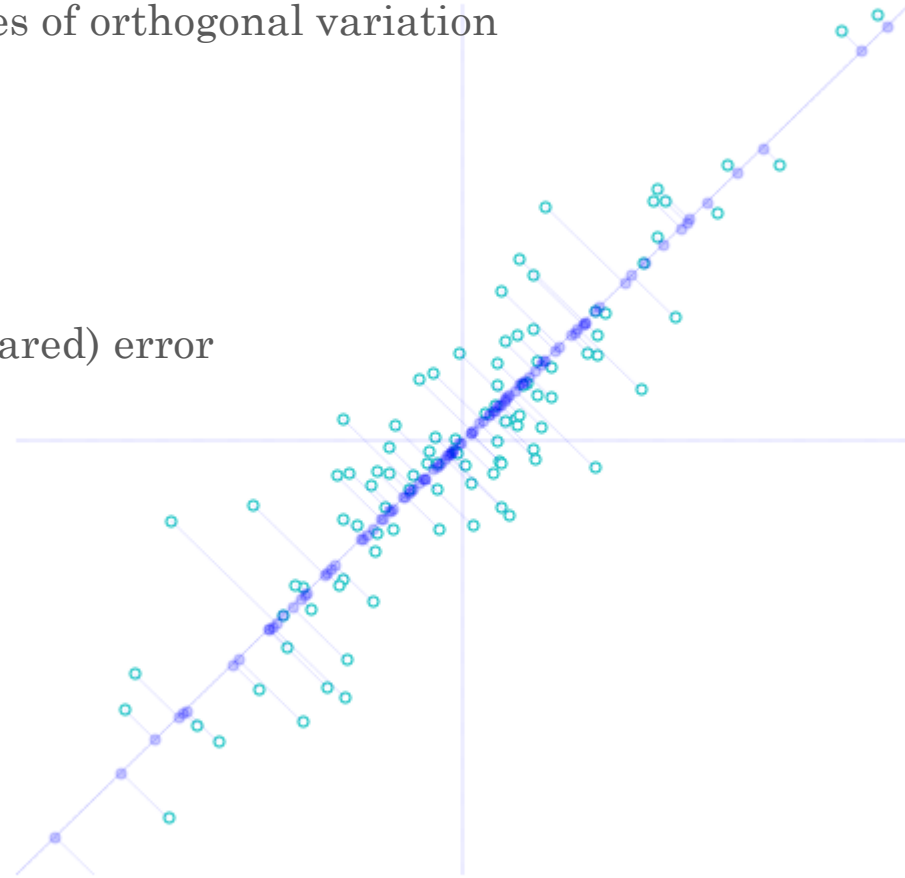
DPCA: intuition

- Classic PCA rotates coordinate system onto axes of orthogonal variation



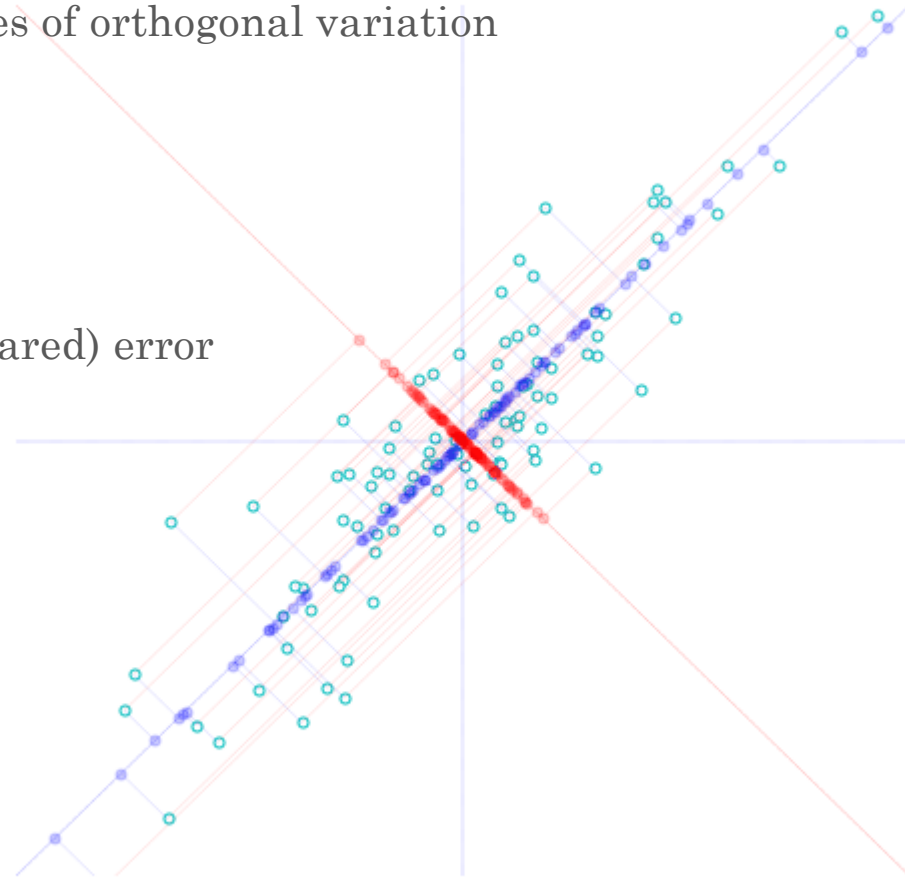
DPCA: intuition

- Classic PCA rotates coordinate system onto axes of orthogonal variation
- And truncates axes of 'small' variation
- Projecting data on axes of major variation
a.k.a principal components
- Reducing dimension with minimum (mean-squared) error



DPCA: intuition

- Classic PCA rotates coordinate system onto axes of orthogonal variation
- And truncates axes of 'small' variation
- Projecting data on axes of major variation
a.k.a principal components
- Reducing dimension with minimum (mean-squared) error
- Fidelity is not what we care about
- Projection distance is not the right metric
- The right metric or 'relevance' depends on what we do with the data

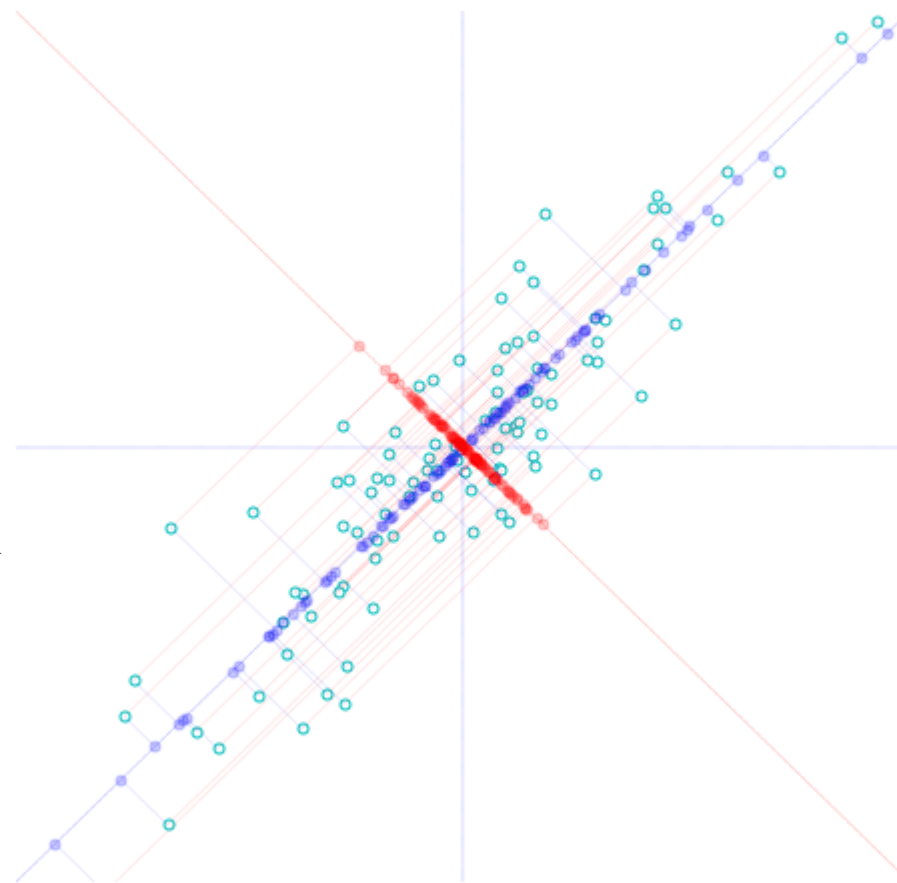


DPCA: intuition

- Suppose we work with a spread option

$$[S_2(T) - S_1(T) - K]^+$$

- Under Gaussian assumption (or approximately otherwise)
 - Depends only on $X_2 - X_1 = S_2(0) - S_1(0)$
 - I.e. coordinate on the **red** axis
- PCA projects data on the **blue** axis
 - Minimizes projection errors
 - But completely misses the risk of the spread option
 - Which is constant along the **blue** axis!

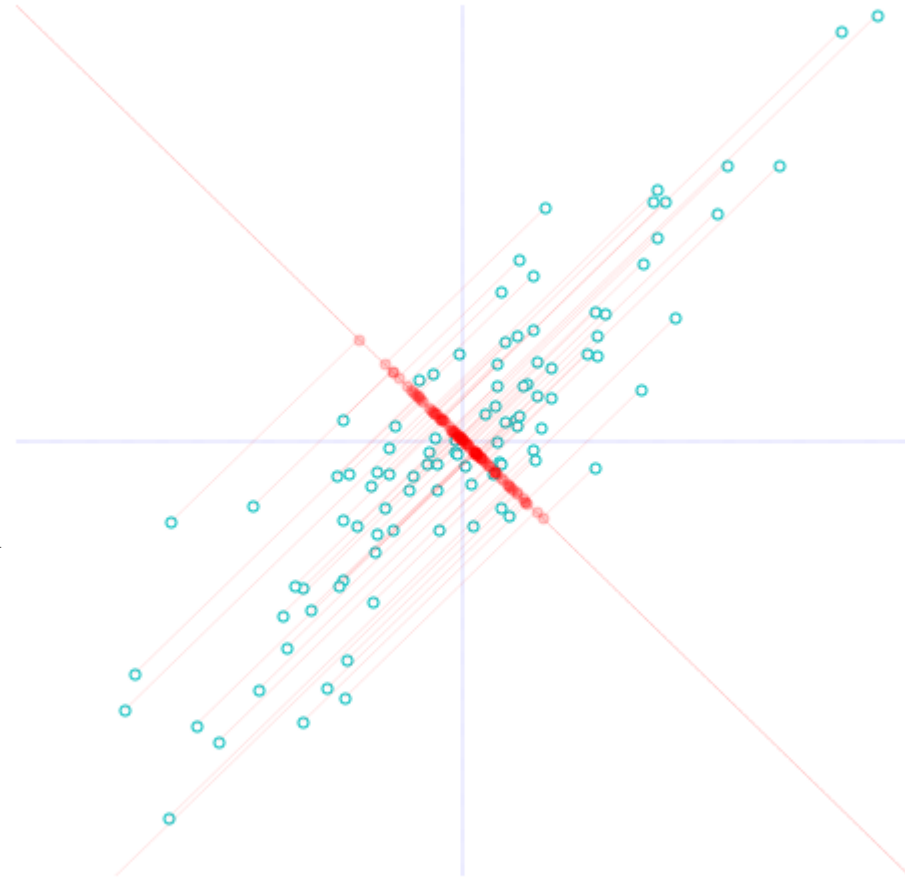


DPCA: intuition

- Suppose we work with a spread option

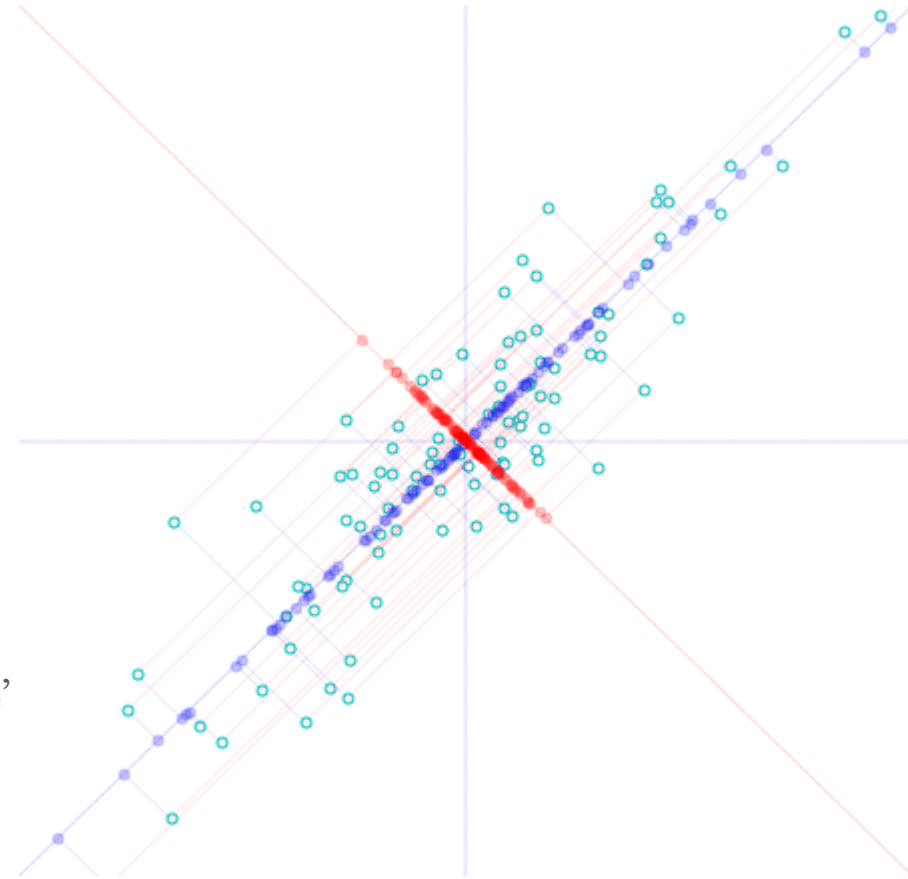
$$[S_2(T) - S_1(T) - K]^+$$

- Under Gaussian assumption (or approximately otherwise)
 - Depends only on $X_2 - X_1 = S_2(0) - S_1(0)$
 - I.e. coordinate on the **red** axis
- PCA projects data on the **blue** axis
 - Minimizes projection errors
 - But completely misses the risk of the spread option
 - Which is constant along the **blue** axis!
- In fact, we can safely truncate the **blue** axis
 - and project onto the **red** axis
 - Projected data may be far away from original data
 - But it is identical for the spread option
- PCA does the opposite!



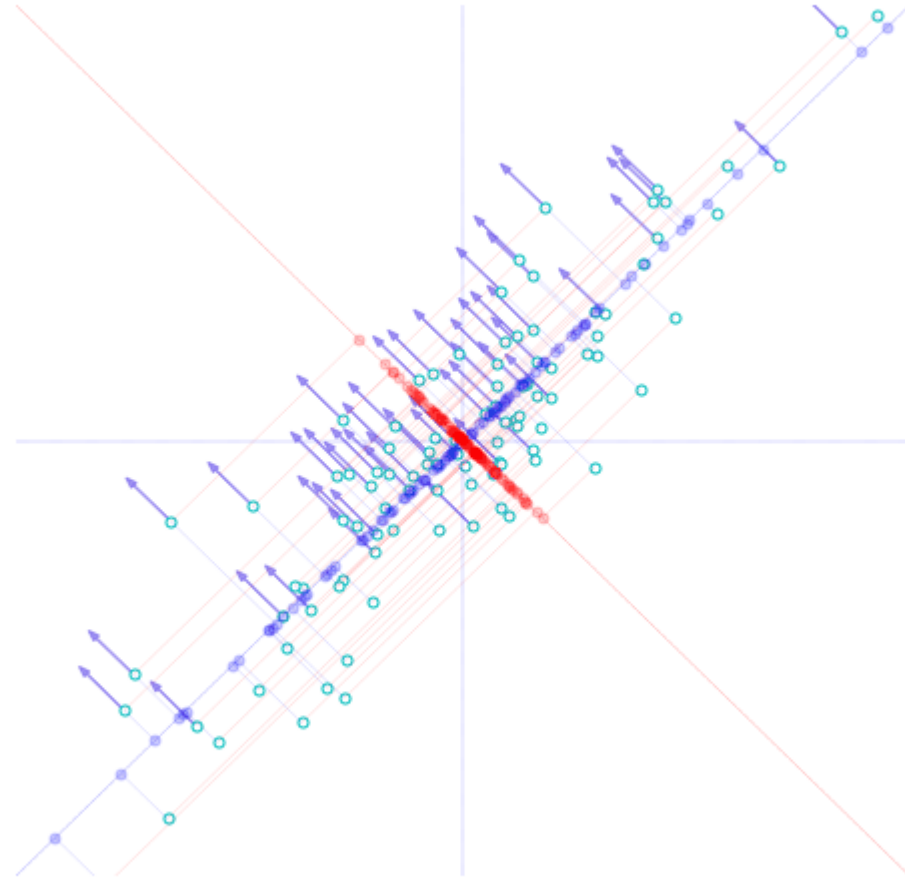
DPCA: intuition

- For the spread option:
 - Variation is concentrated on the **blue** axis
 - Relevance is concentrated on the **red** axis
 - Variation and relevance are orthogonal
- In general, there is no reason why variation should be aligned with relevance
- PCA identifies principal axes of variation
- We want to identify principal axes of relevance
- How can we do that in general?
- We need a mathematical measure of ‘relevance’



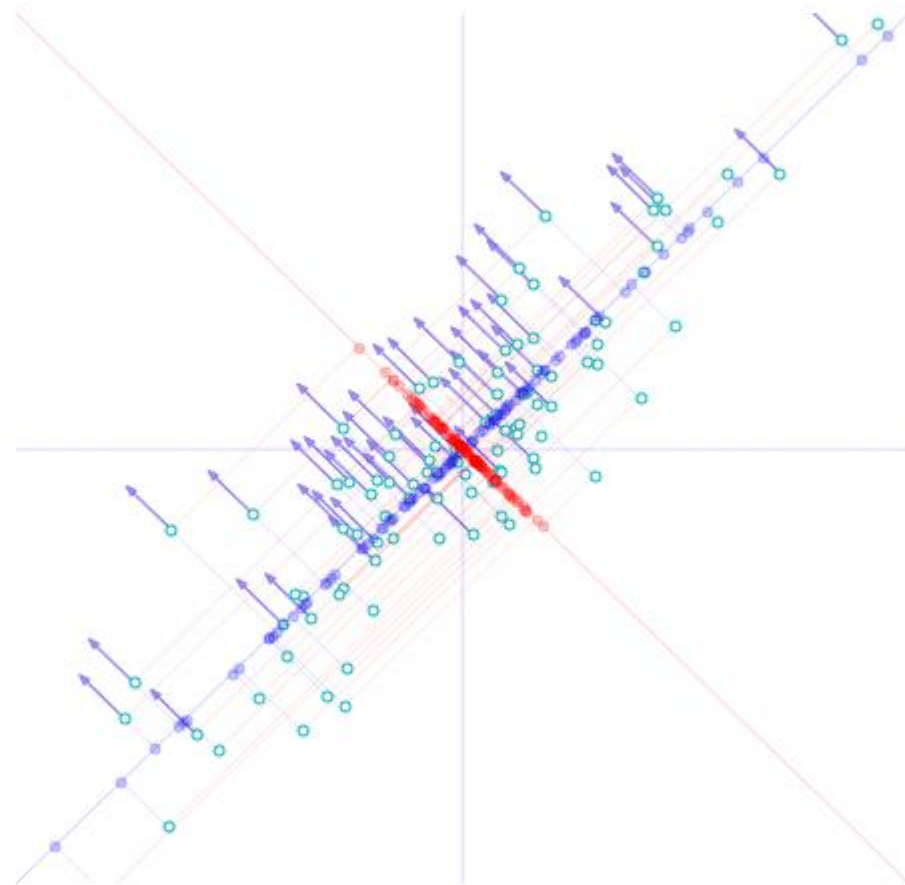
DPCA: intuition

- Relevance is aligned with **gradients**



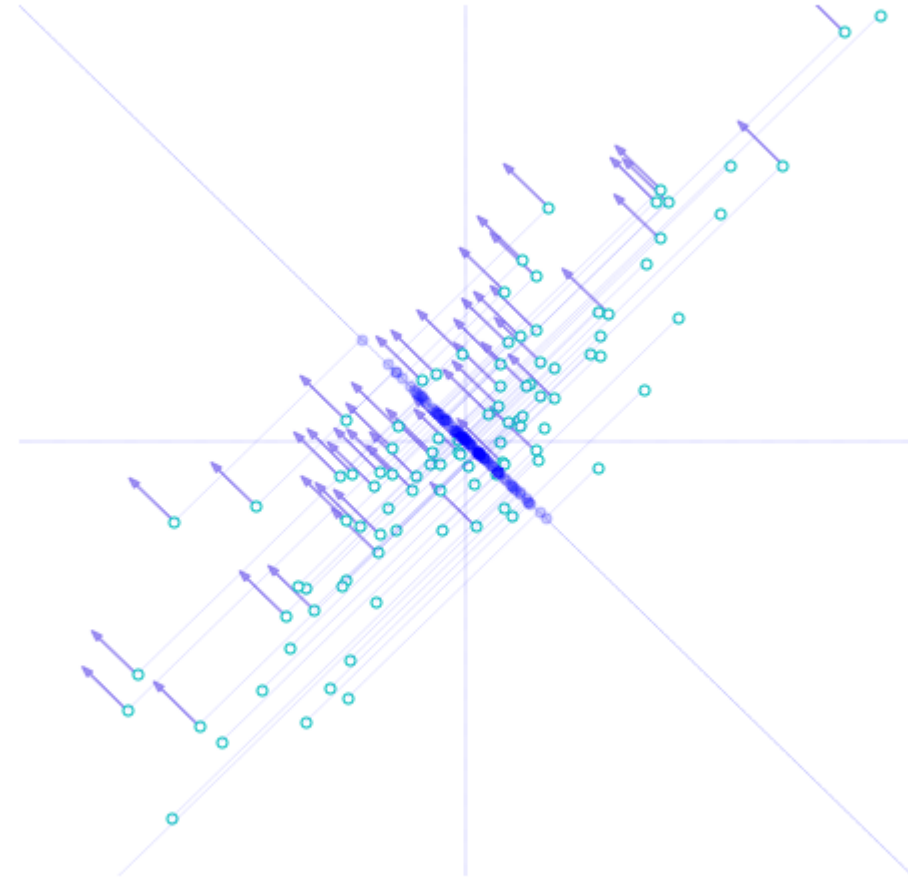
DPCA: intuition

- Relevance is aligned with **gradients**
- So we can align coordinate system in the direction of the gradients



DPCA: intuition

- Relevance is aligned with **gradients**
- So we can align coordinate system in the direction of the gradients
- And safely truncate axes where directional sensitivities are immaterial
- Because the target is (virtually) constant in that subspace



Differential PCA

- Define relevance of axis / subspace by the magnitude of projected gradients:

$$rel(u) = E[(u \cdot Z)^2]$$

- Principal axes of relevance are eigenvectors of:

$$C_{ZZ} = E[ZZ^T] = PDP^T$$

- Relevance is measured by eigenvalues
- Axes / subspaces with low relevance are safely truncated from state representation
- Because payoff h and value v are virtually constant

DPCA: analytic sanity check

- Suppose our problem is one-dimensional in disguise: $h(X; \omega) = f(a^T X; \omega)$
- Then:
$$Z = \frac{\partial h}{\partial X}(X; w) = \frac{\partial f(a^T X; w)}{\partial (a^T X)} a = \underbrace{\gamma(X; w) \|a\|}_{\text{random scalar}} \underbrace{\frac{a}{\|a\|}}_{\substack{\text{deterministic} \\ \text{unit vector}}} = \gamma(X; w) a$$
- Hence: $C_{ZZ} = E[ZZ^T] = \|\gamma\|_2^2 a a^T$ rank 1, only one nonzero eigenvalue $\|\gamma\|_2^2$ with eigenvector a
- Differential PCA correctly identifies from differential data Z :
 - Effective dimension is 1
 - One-dimensional feature is $a \propto a$
 - (Finally) resolving the basket problem in a fully satisfactory manner (and without ANN)

DPCA: analytic sanity check

- More generally, suppose h is (approx.) a function of $p < n$ linear (linearly independent) features of X :

$$h(X; \omega) \approx f(a_1^T X, a_2^T X, \dots, a_p^T X; \omega)$$

- Then (by the same line of reasoning):
 - $C_{ZZ} = E[ZZ^T]$ is of rank p
 - The p eigenvectors with material eigenvalues span the same space as $a_1, \dots, a_p$
- Differential PCA correctly identifies from differential data Z :
 - Effective dimension
 - Relevant factors

DPCA: applications

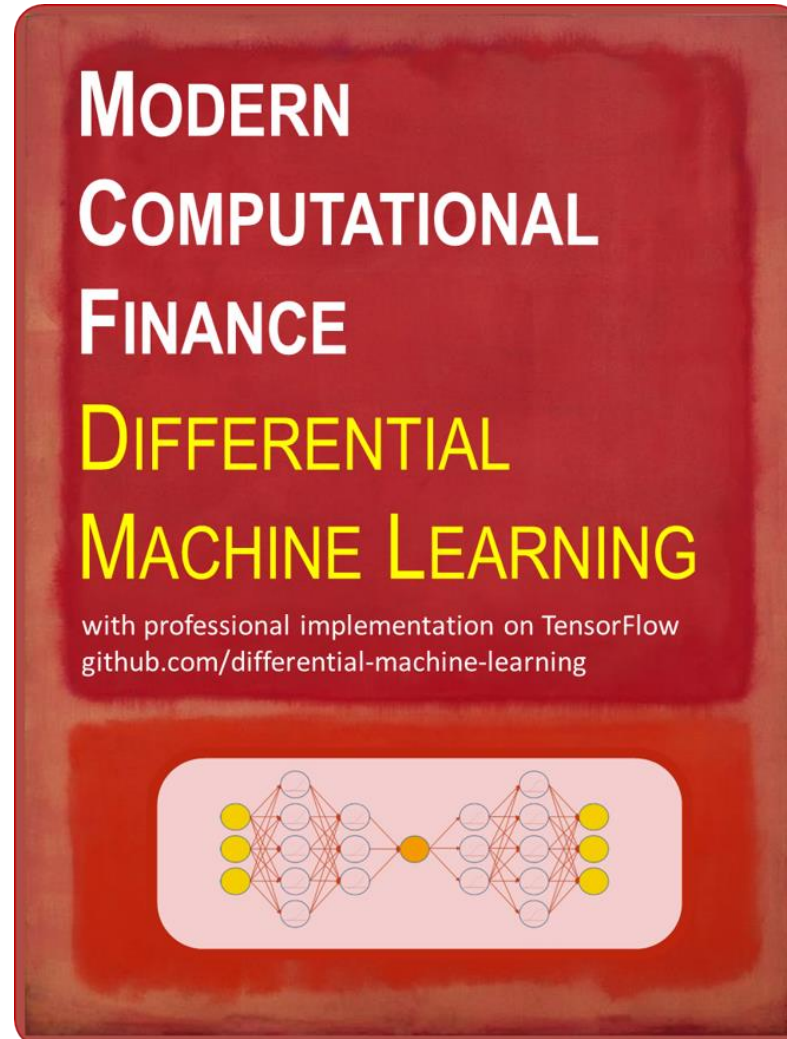
- Safe dimension reduction, effective data compression
- Reliable, lightweight automatic feature engineering for LSM, without neural networks
- Meaningful representation and explanation of risk for complex Derivatives or large trading books
- Computation of risk heatmaps
- Factor dependence of exotic Derivatives or large trading books

<https://github.com/differential-machine-learning/notebooks/blob/master/Bermudan5F.ipynb>



- Even improvement of Monte-Carlo convergence (differential Karhunen-Loeve decomposition)

Printing in 2022 with Chapman & Hall



Thank you for your attention

Superfly Analytics



In-House system of the Year 2015
Winner: Superfly Analytics at Danske Bank

RiskMinds
International

Excellence in Risk Management and Modelling, 2019
Winner: Superfly Analytics at Danske Bank